university of groningen

faculty of science and engineering

Design and Architecture Document

TNO - Analyzer Frontend

S2 Standard

Group Members:

Imad Alihodzic (s5738512)

Panagiotis Panagi (s5065968)

Sertac Bahadir Afsari (s5239885)

Stephanie Radhakrishnan (s5232546)

Teaching Assistant:

Kelian Schekkerman

0. Table Of Contents

0. Table Of Contents 2
1. Introduction 4
1.1. Stakeholders 4
1.1.1. The S2 Consortium 5
1.1.2. Product Owner - TNO 5
1.1.3. Users - Testers And Presenters 5
2. Architecture 6
2.1. Conceptual View 6
2.1.1. Tester And Presenter 6
2.1.2. Python Mock-up Programs 7
2.1.3. S2-Application System 8
2.1.4. Libraries 8
2.1.5. External Services 9
2.2. Container Diagram 10
2.2.1. S2 Analyzer Backend 10
2.2.2. Single-Page Application 11
2.3. Components View 12
2.3.1. Components 13
2.3.2. Pages 13
2.3.3. Parser 14
2.3.4. Models 15
2.3.5. Hooks 16
2.4. Module View 17
2.4.1. Components 17
2.4.1.1. Package “popups” 17
2.4.1.2. Package “messages” 18
2.4.1.3. Package “navbar” 19
2.4.1.4. Package “terminal” 20
2.4.2. Pages 21
2.4.2.1. Package “pages” 21
2.4.3. Parser 22
2.4.3.1. Package “parser” 22
2.4.4. Models 24
2.4.4.1. Package “messages” 24
2.4.4.2. Package “dataStructures” 25
2.4.5. Hooks 25
2.4.5.1. Package “hooks” 26
2.5. Class Diagrams 26
2.5.1. Packages “messages” and “popup”: 28

2.5.1.1. MessageWidget class.... 28
2.5.1.2. MessageCard class.... 29
2.5.1.3. MessagePopUp class.... 30
2.5.1.4. SpecialMessage class.... 31
2.5.1.5. UsageForecastGraph and PowerForecastGraph classes.... 32
2.5.2. Packages “parser”, “terminal”, and “navbar”:.... 32
2.5.2.1. NavBar class.... 33
2.5.2.2. Terminal class.... 33
2.5.2.3. Sidebar class.... 34
2.5.2.4. Parser class.... 35
2.5.2.5. Socket class.... 36
2.5.3. Packages “pages” and “hooks”.... 37
2.5.3.1. Page class.... 37
2.5.3.2. useCloseFilterMenu, useSearch, and useFilters classes.... 38
2.5.4. Package “models.messages” and “models.dataStructures”.... 39
2.5.4.1. MessageHeader and ReceptionStatus interfaces.... 40
2.6. Execution View.... 41
2.6.1. File Selection And Parsing Process.... 41
2.6.2. Real-Time Message Rendering.... 43
2.6.3. Interacting With Messages.... 45
2.6.4. Accessing The Terminal.... 47
2.6.5. Filter Selection And MessageID Search.... 48
3. Technology Stack.... 50
3.1. Typescript.... 50
3.2. Frameworks.... 50
3.2.1. React.... 50
3.2.2. TailwindCSS.... 51
3.2.3. Jest.... 51
3.3. Docker.... 52
3.4. GitHub.... 52
4. Glossary.... 54

1. Introduction

This project involves the development of a frontend to TNO's S2-Analyzer, which is a program that verifies and logs messages exchanged between two devices (namely a Resource Manager and a Client Energy Manager) as defined by the S2 Standard and its FRBC (Fill Rate Based Control) Control Type. As a result, Python mock-up programs mimicking these devices will connect to the backend and send pre-written messages to each other, which get logged alongside any important validation information in a ".txt" file.

As explained in the project's Requirements Document, all of the system's features can be separated into two Epics: The Debugging and the Visual Applications. These act as a collection of requirements that cater to the needs of a specific type of user and outline the project's must, should, could, and will-not haves requirements. To be more specific:

The Debugging Application's purpose is to parse this ".txt" file and translate it into renderable messages based on the JSON models defined by the standard. In addition, it must be able to parse and display any logs regarding the connection status of the communicating devices. All of the above must be rendered in chronological order, adhere to their S2 Standard-defined structure, be accompanied by a status icon which reflects their validity, and be clickable, prompting the appearance of a pop-up window containing additional, not displayed, details.

The Visual Application's purpose is to provide additional features onto the former epic, in order to better suit it for demonstration purposes. Such features include filtering capabilities, message revokeness visualization, and accessibility support. Moreover, it defines different aspects of the appearance of the project.

All in all, both epics take place within the same Single-Page Application which should be deployable via Docker by simply running "docker compose up –build" on the local machine. The Python mock-up programs are expected to be deployed separately.

For the first block, development centered around implementing the then Must-Have requirements, as listed and explained in the first version of the Requirements Document. For the second block, development centered around implementing new features, refactoring, and testing the codebase. Thus, this document will describe the architecture and design that outlines the resulting product.

1.1. Stakeholders

Stakeholders include any person or organization with an interest in, or an effect on, the final outcome of the product, as discussed in the Requirements Document. While their needs and expectations from the project were discussed there, here their architectural concerns are defined as follows:

1.1.1. The S2 Consortium

As the official ambassadors and people responsible for the S2 Standard as a whole, the S2 Consortium is focused on the expandability of the project. This prerequisites the program to be flexible enough to handle future expansions regarding additional features, the support of other S2 Control Types, or the introduction of new types of messages and interactions between the CEM and the RM. Thus, it is crucial that the application can be easily adapted to their needs, and remain future-proof by minimizing its technical debt as much as possible.

1.1.2. Product Owner - TNO

As the Product Owner, TNO is highly concerned with the maintainability and interoperability of the program. On one hand, this means that the program should follow the best design and architectural practices possible, in order to ensure its readability and ease of debugging and maintenance. On the other hand, since the project is dependent on their own backend implementation, it is crucial that the project can seamlessly integrate with TNO's existing S2 ecosystem.

1.1.3. Users - Testers And Presenters

Both testers and presenters prioritize the accessibility, usability, stability, and reliability of the program. This means that the Debugging and Visual Applications should be accessible to people with visual impairments, comprehensible to users or clients of varying technical backgrounds, capable of handling all exchanged messages between the CEM and the RM without running into application-braking errors or bugs, and render their interaction accurately and in detail. All in all, they want to ensure that the project leads to a dependable debugging and presentation tool.

2. Architecture

2.1. Conceptual View

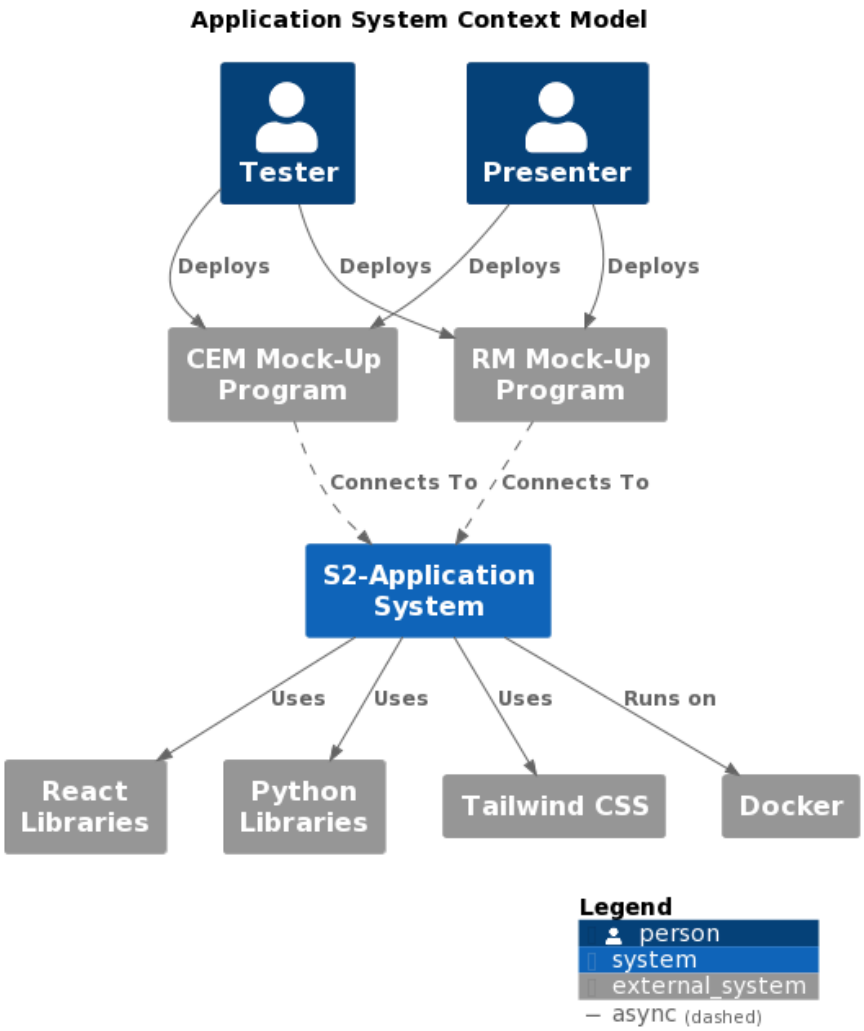

Figure 1: Conceptual View diagram.

2.1.1. Tester And Presenter

- Requirements: *N/A*;
- Description: Hereby referred to collectively as “User”, these are the personas of the Debugging and Visual Application respectively, whose interests and expected actions were discussed in the Requirements Document: On one hand, “Testers” test the correctness of

their S2 Standard implementation and, as such, require to view the different properties of each exchanged message. On the other hand, “Presenters” focus on the demonstration of a pre-defined communication between a CEM and a RM to potential clients and, as such, they need a visually appealing and simple application.

- Goals: They deploy Python mock-up programs in order to test a live or present a pre-established communication scenario between a CEM and RM. The exact goals of these personas are outlined in their counterpart sections in the Requirements Document.
- Input: N/A.
- Output: N/A.
- Design: N/A.
- Changes: N/A.

2.1.2. Python Mock-up Programs

- Requirements: *N/A*;
- Description: Programs written in Python that after connecting to the S2-Analyzer backend, send a pre-established set of messages, mimicking the communication between a CEM and a RM. To be more specific, a Customer Energy Manager (CEM) organizes and schedules the energy consumption of energy-flexible devices by communicating with their Resource Managers (RM).
- Goals: They are the reason behind the S2-Analyzer backend logging a sequence of exchanged messages or connection initiations or closings.
- Input: No input is required, since all the messages are manually programmed in.
- Output: They log every message they receive, along with their connectivity status in the terminal, but this is not important to neither the Debugging nor Visual applications.
- Design: They do not follow any design patterns, since they consist simply of a sequence of asynchronous calls to the S2-Analyzer backend’s websocket, regarding their connectivity status or the exchange of messages. They are very short programs with a clear purpose that exclusively communicate with the backend.
- Changes: N/A.

2.1.3. S2-Application System

- Requirements: *All*;
- Description: The Debugging and Visual applications, along with the S2-Analyzer backend that facilitate all the features and requirements the project must satisfy. In general, the system handles the connection of a CEM and a RM, validates and logs any exchanged messages between them, and displays them in chronological order. It can generate pop-up windows with the properties of each message, show their reception status, mark revoked ones, and filter them.
- Goals: It must be able to provide all the Must-Have requirements, as defined by the Requirements Document.
- Input: The only source of input of the system are the messages exchanged by the CEM and RM Python mock-ups, that arrive via websocket to its backend.
- Output: It logs and renders every message it receives via websocket from both the CEM and the RM, their validation, reception status, their properties, and any additional errors or warnings.
- Design: As will be discussed later, the system is broken off into two containers: The backend and the frontend. As such, it closely follows the Separation of Concerns principle by aggregating and allocating code or similar behavior to the most fitting of the aforementioned containers after a case-by-case analysis. On one hand, the backend comprises Python classes for handling the devices' connections, communication, message schema validation, and logging everything in a ".txt" file. On the other hand, the frontend follows the folder structure established by the React Framework (Figure 3), while also encompassing some logic in the form of the Parser component that reads and parsers the backend's logs and returns them to the frontend as renderable message models. It is able to do this in two ways: either requesting each log via websocket in real-time before it is being written in the ".txt" file or via reading the entire complete ".txt" file the backend produces at once. The decision to incorporate the parser logic into the frontend was one set by the client themselves.
- Changes: The exact changes from the first block will be discussed in detail in the following sections.

2.1.4. Libraries

- Requirements: *US-2-RQ-M-F2*, *US-8-RQ-M-F1*;
- Description: On one hand, the backend uses multiple Python libraries in order to handle logging, type control, connection management, and routing. On the other hand, the frontend

uses multiple React libraries in order to create and render components or generate its file structure. The exact libraries utilized will be discussed in the “Design” point of their respective classes.

- Goals: Among others, in the frontend, these libraries are used in order to render the Terminal-like view of the history logs, the graphs included in specific messages, and the draggable nature of message pop-ups.
- Input: N/A.
- Output: N/A.
- Design: N/A.
- Changes: N/A.

2.1.5. External Services

- Requirements: *US-3-RQ-M-N1*, *US-3-RQ-M-N2*, *US-4-RQ-M-N1*;
- Description: These are the services and frameworks detailed in section 3 that are related to the system’s development, deployment, and maintenance.
- Goals: These external services are used in order to provide structure, improve the readability, maintainability, and development process of the application, while also polishing its appearance.
- Input: N/A.
- Output: N/A.
- Design: N/A.
- Changes: N/A.

2.2. Container Diagram

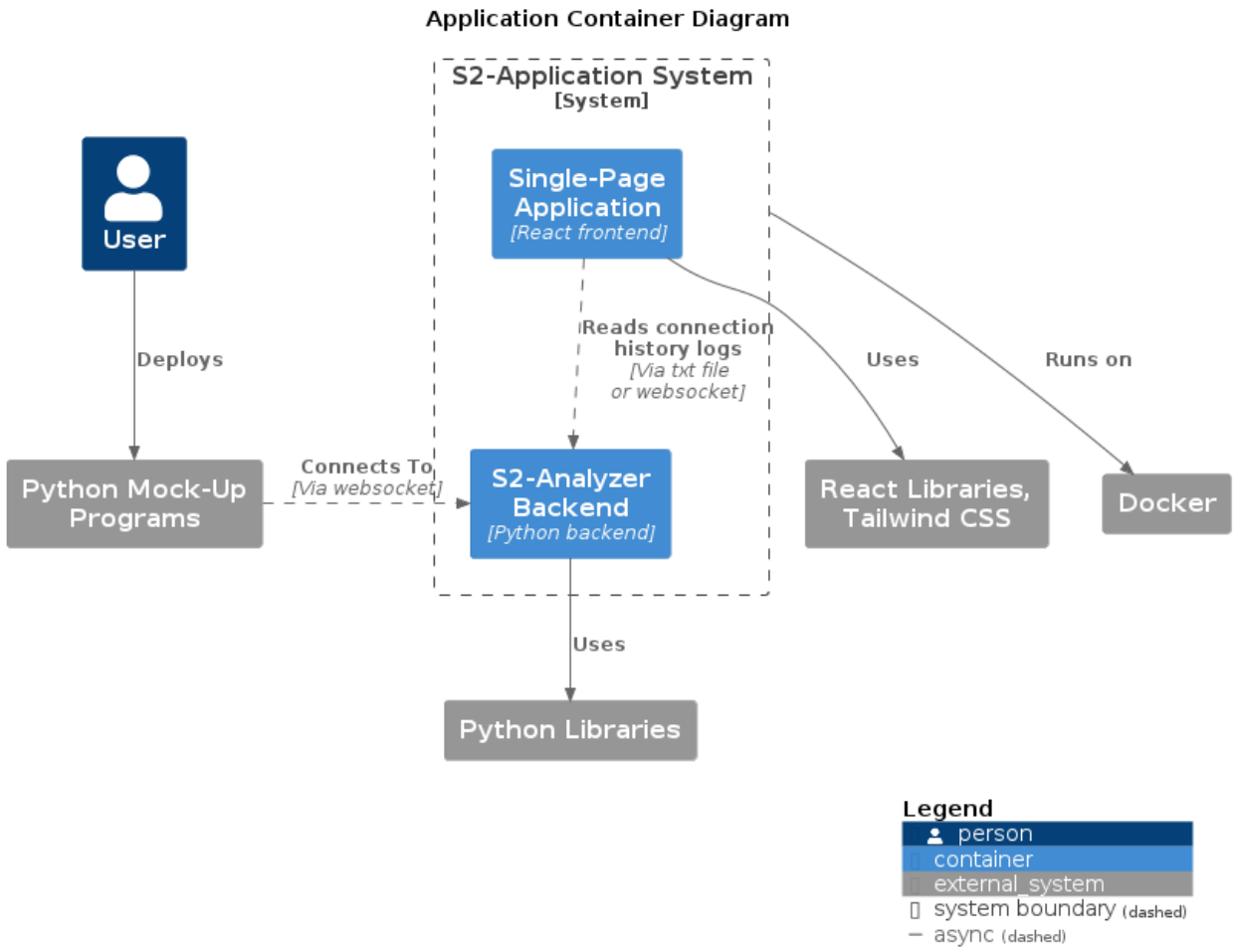

Figure 2: Application Container Diagram.

2.2.1. S2 Analyzer Backend

- Requirements: *N/A*;
- Description: The backend of the application, written in Python by the client company in order to implement and test their S2 implementation.
- Goals: It is responsible for hosting the CEM-RM connection and verifying and logging all their exchanged messages
- Input: It accepts the messages exchanged between the two devices via websocket.

- Output: It creates and maintains the connection's history logs in a ".txt" file and is able to also send each log to the frontend via websocket before writing it.
- Design: While the S2 Analyzer Backend is not the focus of this project, it is still important to note some of its more relevant features: Firstly, it contains a "mocks" package, where all the Python mock-up programs provided by the client reside, which can be used for debugging and testing the frontend. Secondly, it does not support any API for requesting logs or the connection history ".txt" file, so the frontend has to either retrieve each line via websocket before it is being written or allow the user to select and load the entire ".txt" file after its completion. Lastly, since the backend is not meant to support web applications, it does not handle communications via HTTP commands, but rather all connections are handled via websockets.
- Changes: N/A.

2.2.2. Single-Page Application

- Requirements: ***All Must-Haves***;
- Description: The container responsible for parsing and rendering all exchanged messages between the CEM and RM.
- Goals: It must fulfill all Must-Have requirements, such as rendering every exchanged message between the CEM and the RM, creating a pop-up window with the properties of such a message if it was clicked, and showing the revokeness or reception status of each of them, among others.
- Input: The SPA accepts input in the form of user-generated events, such as mouse clicking or dragging. Also, it receives new messages to render by calling the respective function of its Parser component that is able to connect to the backend, but more details on that will be discussed later.
- Output: Renders the entire communication between the CEM and RM in chronological order and according to the selected filters.
- Design: This application follows the React project structure, separating independent objects as components, which can be interconnected or aggregated in larger views, namely "pages". As such, it boosts the overall maintainability and expandability of the system by building the user interface from the bottom-up, beginning with small independent components and ending with large composite pages. In addition, it forces data to be transferred one-way from the head of the DOM tree to the bottom, making debugging much easier and forcing developers to think about their code structure and design before implementing it.

- Changes: Among others, it has incorporated new features such as real-time message reading from the backend, an expansive filters menu, and more specialized message renderings. Also, its appearance was modernized and made use of TailwindCSS' visual and animation capabilities.

2.3. Components View

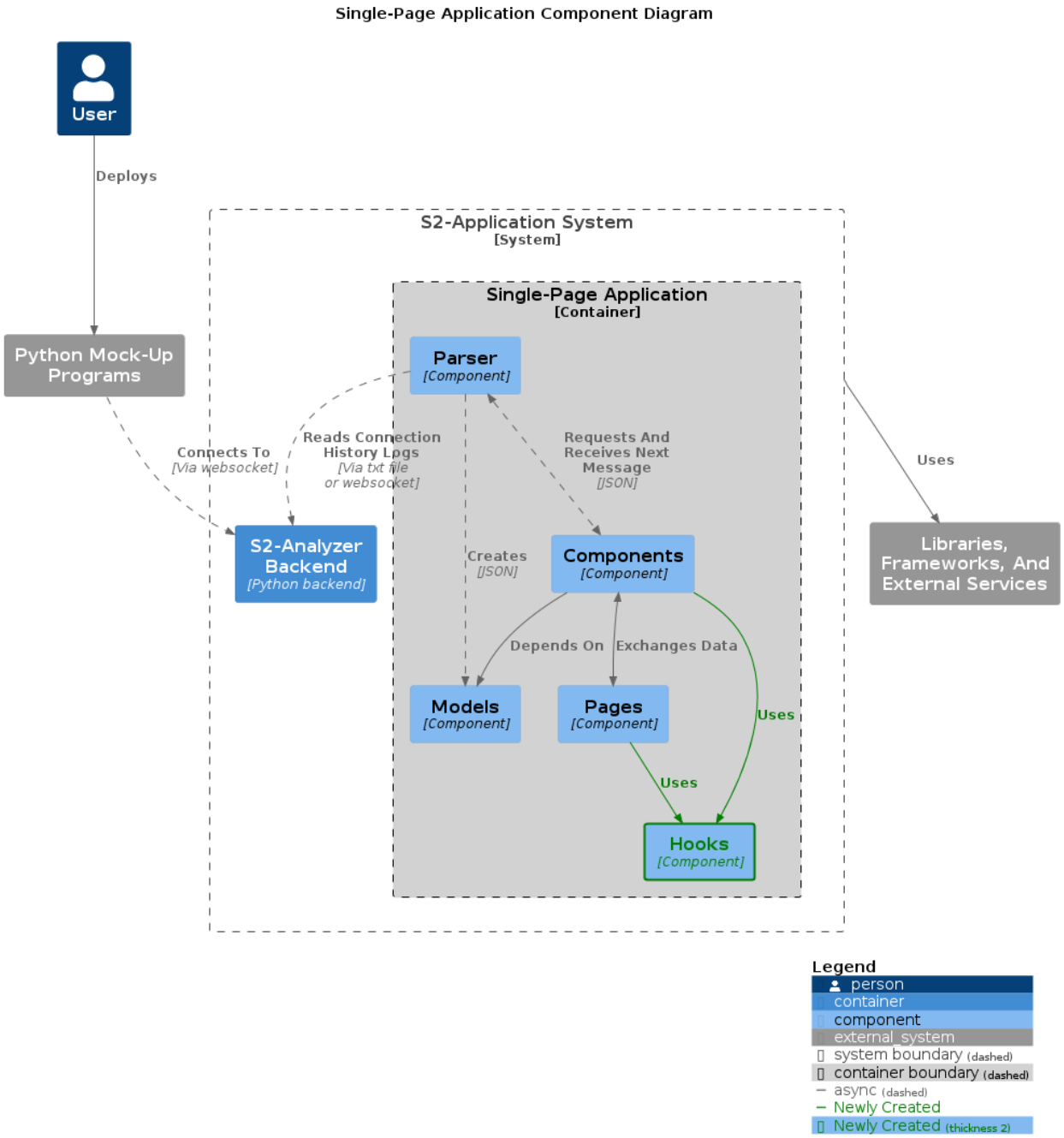

Figure 3: Single-Page Application Component Diagram.

2.3.1. Components

- Requirements: *US-1-RQ-M-F1*, *US-1-RQ-M-F2*, *US-1-RQ-M-F3*, *US-2-RQ-M-F1*, *US-2-RQ-M-F2*, *US-5-RQ-M-F1*, *US-5-RQ-M-F2*, *US-5-RQ-M-F3*, *US-6-RQ-M-F1*, *US-7-RQ-M-N2*, *US-7-RQ-M-F1*, *US-8-RQ-M-F1*, *US-8-RQ-M-F2*, *US-9-RQ-M-F1*, *US-9-RQ-M-F2*, *US-10-RQ-M-F2*, *US-10-RQ-M-F3*, *US-11-RQ-M-F1*, *US-11-RQ-M-F2*, *US-13-RQ-M-N1*, *US-13-RQ-M-N2*, *US-14-RQ-M-F1*;

- Description: This component refers to the collection of reusable and mostly independent visual components that are aggregated by the Pages component in order to create large-scale User Interfaces.

- Goals: It provides reusable and independent TSX components for message rendering, pop-up creation, updating the view according to the connection history logs, and customizing the navigation bar.

- Input: This varies from component to component, but generally the message models created by the Parser component are the cornerstone of the Components component and its encompassing classes.

- Output: Each member of this component is responsible for rendering a specific aspect of the User Interface, whether that refers to individual messages, validation statuses, the navigation bar, the communication timeline, etc.

- Design: In general, this component follows the Composite and Decorator Design patterns, with many of its classes either being independent or using other independent ones, in order to build their behavior around them. This design choice is very common amongst frontend systems and boosts flexibility, extendability, and readability, while promoting a strict Separation of Concerns that is crucial when debugging.

- Changes: Existing classes were refactored, the component's structure was reorganized, a connection was added between this and the Hooks component, and new packages and classes were created in order to facilitate the system's new features. More details will be discussed in sections 2.4.1 and 2.5.

2.3.2. Pages

- Requirements: *US-1-RQ-M-F1*, *US-1-RQ-M-F2*, *US-3-RQ-M-N1*, *US-3-RQ-M-N2*, *US-4-RQ-M-N1*, *US-7-RQ-M-N1*, *US-12-RQ-M-N1*, *US-13-RQ-M-N1*, *US-13-RQ-M-N2*;

- Description: Aggregations of the elements defined in the Components component in order to create User Interfaces catered to the requirements of the system's users.

- Goals: It handles user input and percolates such events and data transactions throughout the related components.
- Input: It accepts user-generated events, such as mouse clicks and mouse dragging.
- Output: It reflects user-intended changes in their related components.
- Design: This component follows the Composite Pattern, since its dependencies on other classes and the data-flow between them, follows a tree-like structure. As such, classes defined within the Pages component do not hold a significant state or behavior, but instead rely on individual classes to handle specific events and actions. This allows for easier customization of the different aspects of the user experience, boosting maintainability and expandability.
- Changes: Some of the behavior exclusive to the Pages component was extracted into its own component, namely the Hooks component, in order to make the expandability and maintenance of the system as smooth as possible.

2.3.3. Parser

- Requirements: *US-2-RQ-M-F3*, *US-5-RQ-M-F3*, *US-6-RQ-M-F2*, *US-8-RQ-M-F2*, *US-10-RQ-M-F1*, *US-10-RQ-M-F2*, *US-10-RQ-M-F3*, *US-11-RQ-M-F1*;
- Description: The component responsible for accessing the backend's history logs and parsing any new messages that were exchanged.
- Goals: It separates the logic of reading and parsing files into its own component, while providing the ability to return all parsed messages to the frontend after some processing (for example, it is able to update all messages when an update is given, a ReceptionStatus or RevokeObject message has been sent referencing the original, etc).
- Input: It reads a ".txt" file that contains the connection history logs of the CEM-RM communication or it accepts that file line-by-line as it is being written in real-time.
- Output: It returns the list of messages that have been parsed from the connection history logs or raises an error if needed. It returns them in the form of renderable models as defined by the Models component.
- Design: The Parser component contains pure Typescript (not TSX) files since it encompasses all parsing logic in one Parser class and all connection-management logic in one Socket class. The latter makes use of the former's public functionality in order to pass logs from the backend to the frontend and have them be transformed and saved as frontend models. The most important public functionality the Parser class provides is the following:

1. parseLogFile(): A method that begins the process of file selection, line parsing, message processing, and in the end, returns the list of resulting messages. File selection here happens manually - from a file explorer window where the user is able to select which ".txt" file(s) they wish to be parsed. In the case of multiple ".txt" files, their contents will be concatenated.

2. getLines(): A method that returns the lines of the parsed file in the form of a continuous string. No parsing or processing is done, since this method is only used for providing the input of the Terminal class, as will be discussed later.

3. parse(s:string): A method that parses either a single line of text or multiple lines separated by new line characters. This was previously a private method that was used only within the parseLogFile() method, but has since become public in order to get called by the Socket class on every message that is exchanged in real-time between the CEM and the RM.

- Changes: A new class was added to the component, namely the Socket class, whose purpose is to receive all exchanged messages from the S2 Analyzer's backend websocket in their logged form and pass them to the Parser by calling its parse(s:string) method. More about the exact functionality of these classes will be discussed in their respective class-level sections.

2.3.4. Models

- Requirements: *US-2-RQ-M-F1*, *US-2-RQ-M-F3*;

- Description: The frontend representations of the messages defined by the FRBC Control Type of the S2 Standard.

- Goals: It presents accurate models that follow robust OOP principles in order to represent the different message entities and their comprising data structures.

- Input: N/A.

- Output: N/A.

- Design: While the exact structure of these models will be discussed in the following section, it is important to note that they were created based on strong Object Oriented principles. This means that the common properties of all messages that are the minimum required ones for rendering them to the user, are extracted as a parent interface to all messages. Thus, it is much easier to parse messages according to their type, reuse classes between different types of messages, and provide a clear outline of how to introduce a new type of message to the system.

- Changes: None.

2.3.5. Hooks

- Requirements: *US-9-RQ-M-F1*, *US-9-RQ-M-F2*;
- Description: A supporting component who contains functionality that will either be shared between multiple components or is better separated into its own utility classes.
- Goals: It aims to increase the Separation of Concerns of already complicated classes by extracting some behavior in utility classes. Thus, it aspires to provide a clear interface of what to alter or expand upon by future developers of the system regarding important functionality, such as the filtering capabilities of the application.
- Input: It differs between hook implementations, but will be discussed in more detail in the following sections.
- Output: Similarly, it differs between hook implementations, but it is expected that these classes do not output anything to the user. Instead, they are used for some internal processing, the results of which will be used and rendered by other classes.
- Design: Members of this component follow the expected hook structure enforced by React that involves implementing and exporting a specific “function”. Thus, by importing any of them, they can be used as function calls in order to perform some otherwise heavy or complicated internal computations needed by other classes of the system. Since all members are utility classes, they do not hold any significant state apart from local variables and are, thus, considered lightweight classes that can be used in a wide variety of scenarios without significant costs.
- Changes: This component as a whole was created after refactoring the frontend’s architecture from the first block. It aims to make extensive use of React’s capabilities and be a step towards model React systems that are characterized by their expandability and maintainability.

2.4. Module View

2.4.1. Components

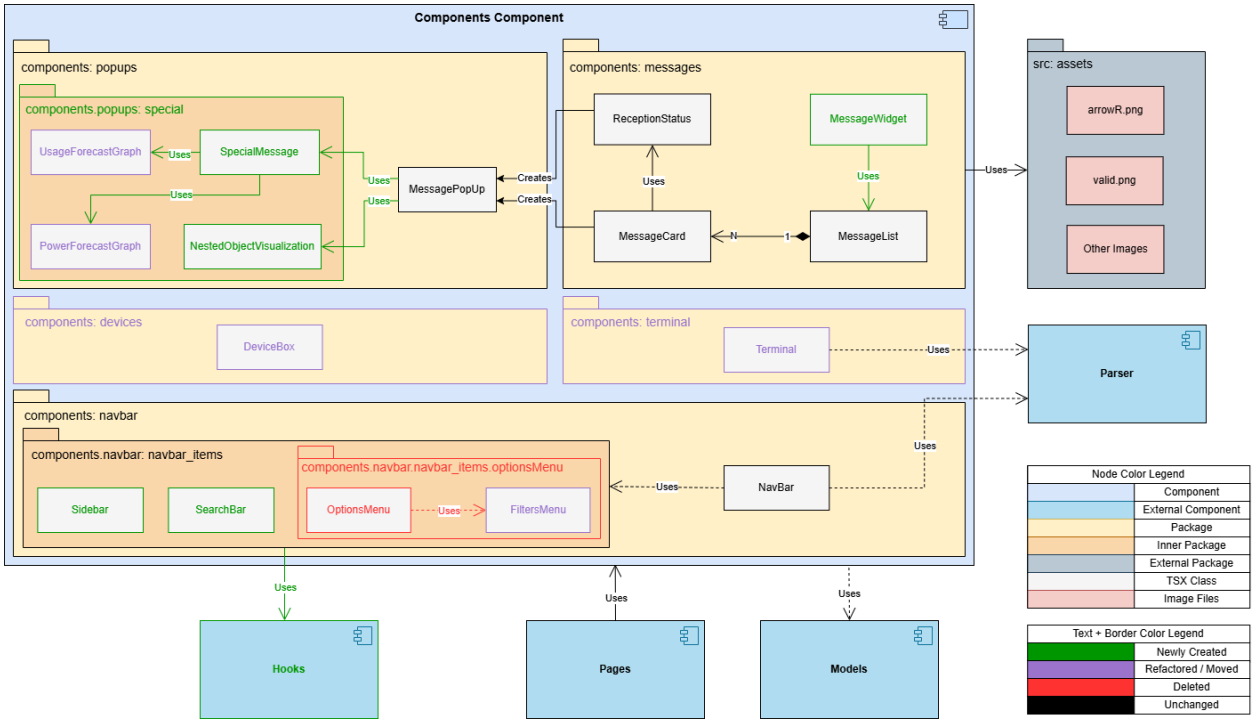

Figure 4: Single-Page Application's Components Module Diagram.

2.4.1.1. Package "popups"

- Requirements: *US-2-RQ-M-F1*, *US-2-RQ-M-F2*, *US-13-RQ-M-N2*;
- Description: The package that contains all classes related to the message's pop-ups.
- Goals: It provides a reusable way of presenting each message's properties across all types, while still catering to the PowerForecast, UsageForecast, and SystemDescription messages which need special attention (i.e. rendering their elements in a graph or in a collapsable way, respectively).
- Input: The access point of the whole package is the MessagePopUp class which is able to display the properties' names and values of each message, regardless of type, in a property-value table. It accepts the message that is supposed to be rendered, a boolean trigger that denotes if the pop-up should be visible or not, and a setter method for that.
- Output: It renders a pop-up with the properties of each message in a property-value table. More specifically, it graphs the elements of the two aforementioned "forecast" messages and

it adds buttons to the table's rows in order to make them collapsible when rendering a SystemDescription message.

- Design: Instead of creating multiple pop-ups for each type of message, this package is able to define only a single class (the MessagePopUp class) that caters to any children of the MessageHeader class, which all messages extend. Moreover, it is able to utilize the customized-rendering related classes in the "special" package in order to incorporate their functionality into every pop-up that they are needed in. It is able to do so via the SpecialMessage factory class that acts as a gateway to the customized rendering of each such message. Thus, the package follows the Composite Pattern and achieves a high Separation of Concerns, maintainability, and reusability.

- Changes: As observed in Figure 4, the classes related to the customized rendering of messages were aggregated into their own new package called "special". In addition, the SpecialMessage class was created in order to return the specific class needed for rendering the type of message that each MessagePopUp is currently holding, if needed. Please note that the NestedObjectVisualization class (which was created in order to cater to the incredibly long and nested nature of the SystemDescription message) can be accessed independently of the SpecialMessage class. This is because this class, unlike the others, is the only one that affects the property-value table of the MessagePopUp, instead of simply appending new information beneath it. Thus, creating a separate "SpecialTable" factory class only for this was deemed too cluttering, since it is unlikely that any current or possibly future-added type of message will need to alter its property-value table in any different way than the one offered by the reusable NestedObjectVisualization class.

2.4.1.2. Package "messages"

- Requirements: *US-1-RQ-M-F1*, *US-1-RQ-M-F2*, *US-1-RQ-M-F3*, *US-5-RQ-M-F1*, *US-5-RQ-M-F2*, *US-5-RQ-M-F3*, *US-6-RQ-M-F1*, *US-7-RQ-M-F1*, *US-10-RQ-M-F2*, *US-13-RQ-M-N1*, *US-14-RQ-M-F1*;

- Description: The package that contains all classes related to rendering messages.

- Goals: This is the package responsible for rendering every type of message, aggregating all of them in a scrollable list, and denoting next to them their reception status.

- Input: The entrypoint of this package is its MessageWidget class that is used directly by the Pages Component in order to use a MessageList to render multiple MessageCard instances in a visually appealing and scrollable manner. Thus, the MessageWidget class only requires an array of MessageHeaders that will be passed onto the MessageList.

- Output: It renders the common properties of all messages (more specifically, their type, date-time of arrival, sender, and receiver) alongside an icon which resembles the reception status of each message. Thus, the MessageWidget class aggregates each MessageCard within a MessageList and adds more helpful visual details in order to clearly demonstrate the CEM-RM communication.

- Design: Similarly to the “popups” package, this one was built around reusability principles alongside the Composite Pattern. From the MessageList class encompassing multiple MessageCards to each of them containing a ReceptionStatus and a MessagePopUp, this package follows a simple tree-like structure with the MessageWidget class acting as a Decorator class that is its root and the entry-point to the whole package. Moreover, the MessageCard class uses image files to render the different kinds of arrows used throughout the MessageList’s Sequence-Diagram-like appearance, while the ReceptionStatusIcon class uses images to denote the reception status of each message as follows: A green checkmark for valid messages, a gray dot spiral for buffered messages, a warning red triangle for errors, and a white twisted arrow for revoked messages.

- Changes: The biggest change of this package is the creation of an additional level of abstraction on top of the MessageList class, namely the MessageWidget. This class integrates the behavior of the previously-present DeviceBox (hence that package and class being marked as “refactored” in Figure 4) used for rendering the borders of the Sequence-Diagram-like structure of the CEM-RM communication. By combining this with the MessageList, it is able to create a reusable widget capable of fully rendering a CEM-RM communication. This vastly boosts the expandability of the application, since due to the modular nature of the MessageWidget class, it is possible to visualize a “one CEM - multiple RMs” communication by simply instantiating a MessageWidget for each RM present. Thus, it establishes the longevity and usefulness of the application.

2.4.1.3. Package “navbar”

- Requirements: *US-7-RQ-M-N2*, *US-9-RQ-M-F1*, *US-9-RQ-M-F2*, *US-10-RQ-M-F3*, *US-11-RQ-M-F1*, *US-11-RQ-M-F2*;

- Description: The package that contains all classes related to the navigation bar of the application.

- Goals: It provides a simple and extendable way of presenting the different features of the application via buttons the user can select.

- Input: This package’s classes handle user-generated events, such as mouse clicks, on the different buttons and menus the navigation bar contains.

- Output: It renders the navigation bar of the application, alongside any buttons or extendable menus that offer the different features of the application to the user.
- Design: This package heavily relies on the Composite Pattern by having independent classes defined in a concern-related sub-package that the NavBar class can easily incorporate in order to offer different features to the user. Thus, this package is easily expandable, readable, and maintainable - aiming to be simple and comprehensible.
- Changes: Firstly, two new classes were introduced to the “navbar_items” sub-package in order to handle visualizing the parsing errors produced by loading a “.txt” file and searching for a specific messageID, respectively. In addition, the FiltersMenu class that previously resided in an even more inner sub-package called “optionsMenu” was brought up a layer due to the abolishment of the idea of it being integrated into an OptionsMenu class, since that would clash with the simple and “one-click-away” nature of the application. Thus, instead of being hidden behind another menu, the FiltersMenu will be accessible directly through the navigation bar and the OptionsMenu class was deleted. Lastly, the “terminal” subpackage that was previously located in “navbar_items” was extracted into its own package due to the fact that after the modernization of the application’s UI, the terminal-like view became independent of the navigation bar, as will be discussed in section 2.4.1.4.

2.4.1.4. Package “terminal”

- Requirements: *US-8-RQ-M-F1*, *US-8-RQ-M-F2*, *US-13-RQ-M-N1*, *US-13-RQ-M-N2*;
- Description: The package that contains all classes related to rendering the Terminal-like view of the communication’s history logs.
- Goals: It aims to offer a smoother way of transitioning from the previously exclusively console application view to the new UI by allowing its users to compare the visualization of each message to the way they were used to before.
- Input: The sole input of this package comes from the Parser component in the form of the aforementioned “getLines()” method. This returns a continuous string form of the backend’s logs that led to the messages currently being rendered.
- Output: It renders the backend’s string logs in a terminal-like manner.
- Design: Splitting the Terminal from the navigation bar has allowed this package to become very independent, drastically reducing its coupling with other classes as it now only depends on the Parser. Thus, the modularity of this package and its encompassing class increases the maintainability of the application, allowing for effort-free debugging.

- Changes: As mentioned in section 2.4.1.3, this package was extracted from “navbar_items” since block 1. This happened due to the fact that after the modernization of the application’s UI, the terminal-like view was allocated a permanent spot on the bottom of the screen, making it completely independent of the navigation bar and increasing its importance as a visual element of the application since it is constantly visible.

2.4.2. Pages

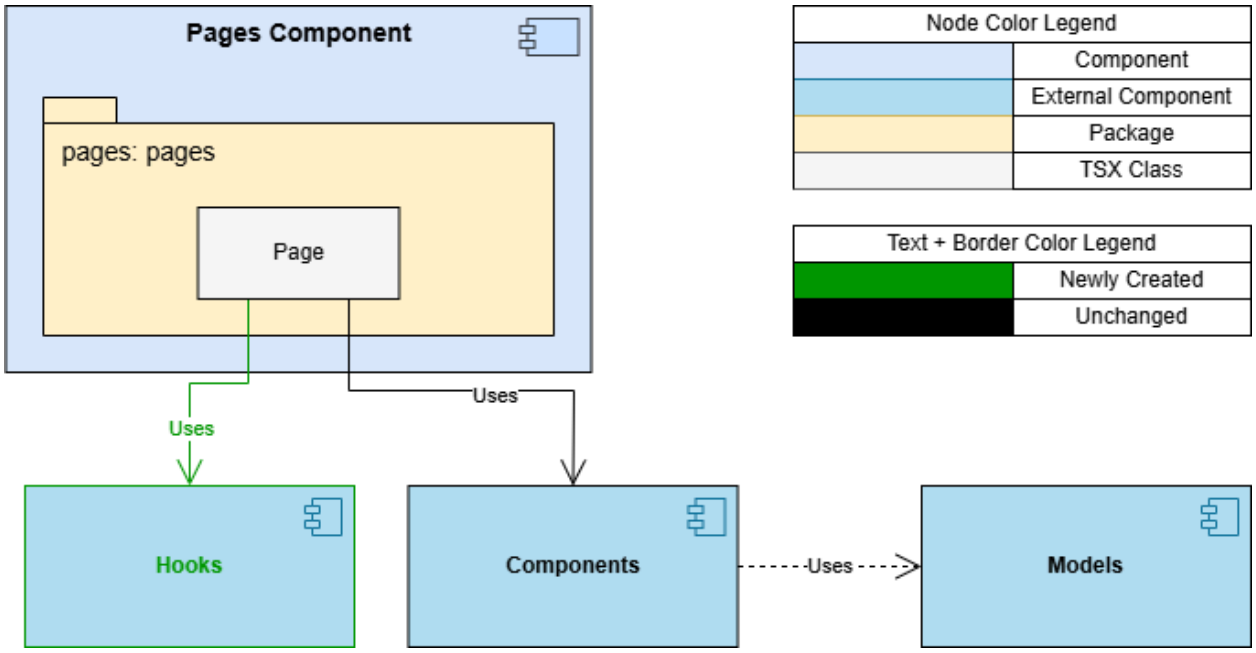

Figure 5: Single-Page Application’s Pages Module Diagram.

2.4.2.1. Package “pages”

- Requirements: *US-1-RQ-M-F1*, *US-1-RQ-M-F2*, *US-3-RQ-M-N1*, *US-3-RQ-M-N2*, *US-4-RQ-M-N1*, *US-7-RQ-M-N1*, *US-12-RQ-M-N1*, *US-13-RQ-M-N1*, *US-13-RQ-M-N2;*
- Description: The package that contains all the pages of the application.
- Goals: It provides large-scale conglomerates of components in a visually appealing way, without relying on knowing how they function.
- Input: Pages in general do not accept any kind of input since they represent the Graphical User Interface of the application. All user-generated events, such as mouse clicking, movements, or dragging are handled by the individual components themselves.
- Output: It renders the complete Graphical User Interface of the application, which in this case refers to a single page.

- Design: This package is the cornerstone of the Composite Pattern used throughout the application, as it quite literally consists of independently-functioning components, who together offer all the features of the Debugging and Visual applications. Thus, this component is only dependent upon the “Components” one, making it indirectly dependent on “Models” in order to host the MessageHeader array that propagated throughout the related components.

- Changes: Since block 1, the Hooks component was introduced as a way of extracting long and complicated methods from the Page class into their own utility classes in order to further promote the Separation of Concerns, maintainability, and flexibility. More specifically, the extracted behavior revolved around handling the filtering and messageID-searching capabilities of the application that would otherwise needlessly overcomplicate the Page class.

2.4.3. Parser

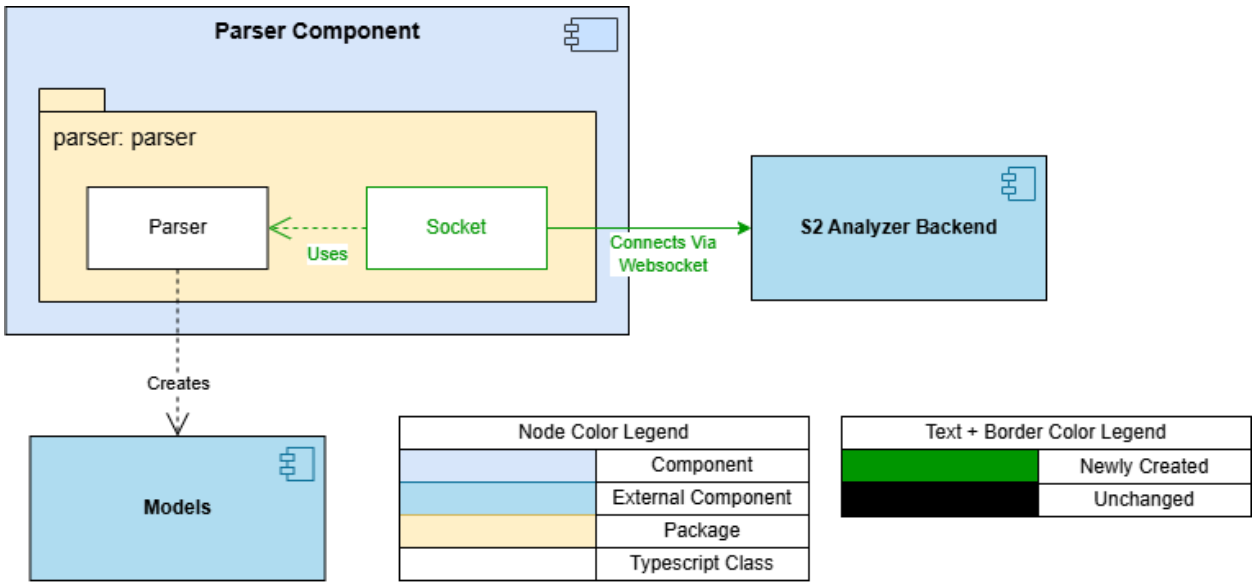

Figure 6: Single-Page Application’s Parser Module Diagram.

2.4.3.1. Package “parser”

- Requirements: *US-2-RQ-M-F3*, *US-5-RQ-M-F3*, *US-6-RQ-M-F2*, *US-8-RQ-M-F2*, *US-10-RQ-M-F1*, *US-10-RQ-M-F2*, *US-10-RQ-M-F3*, *US-11-RQ-M-F1*;

- Description: The package that contains the parsing and connection-handling logic of the application.

- Goals: It provides an easily-extendable way for parsing ".txt" files and allocating their logs to renderable models.
- Input: The parser either accepts a file that it translates into a continuous string, later split on "\n" characters in order to be parsed line-by-line, or it accepts these lines one by one as they arrive at the Socket.
- Output: It does not render anything, but instead is able to return the continuous string form of the inputted or constructed file and an array with its parsed messages.
- Design: As per the client's request, this package and its logic was incorporated into the frontend and it contains pure Typescript classes whose design was based around their functionality. On one hand, since the Parser class provides the input of many other components, it abides by the Singleton Pattern in order to reduce its object instantiation to a bare minimum and increases modularity, testability, and error-tolerance. On the other hand, the newly added Socket class is a very simple one that contains only two methods: one for sending messages to the backend's websocket (only used when first establishing the connection) and one to receive messages from it. Thus, its simplicity boosts the readability and maintainability of the system.
- Changes: As mentioned above, after the Dockerization of the application, the Socket class was created in order to connect the frontend to the backend and read its logs in real-time. Since both of these classes encompass the entirety of the frontend's logic, they were extensively Unit Tested, as observed by the "tests" folder within the project's file structure.

2.4.4. Models

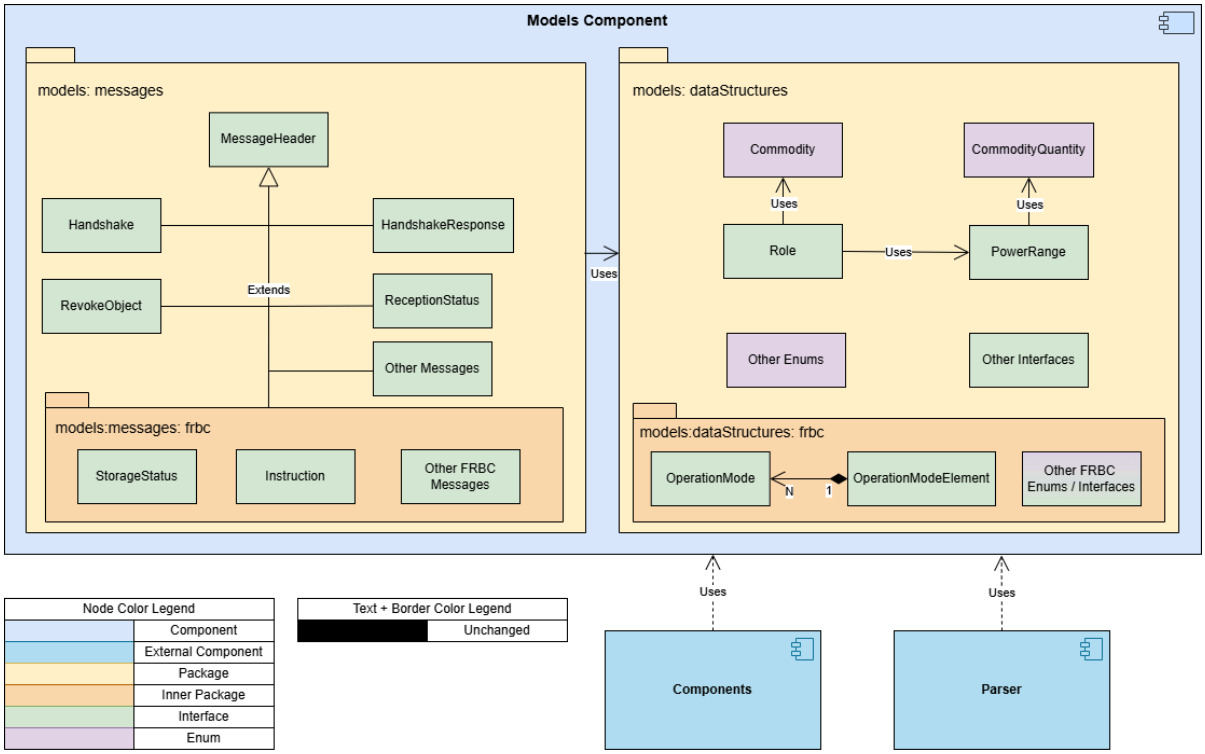

Figure 7: Single-Page Application's Models Module Diagram.

2.4.4.1. Package "messages"

- Requirements: *US-2-RQ-M-F1*, *US-2-RQ-M-F3;*
- Description: The package that contains all the frontend representations of every message that can be exchanged between the CEM and the RM.
- Goals: It provides an accurate and complete description of the properties defined within each message type, as defined by the S2 Standard, while abiding by Object Oriented principles.
- Input: N/A.
- Output: N/A.
- Design: This package contains pure Typescript interfaces. Moreover, in order for the frontend to be able to render all of these different message types, while still retaining their unique properties, the models were made to follow a strong Object Oriented structure, where their common properties (the ones rendered via the MessageCard class) are extracted into their own interface named MessageHeader. Thus, extending the set of

accepted messages is simple and all components can use generics to accept any children of this class to center their functionality around, while still being able to access their exact fields when needed to render a MessagePopUp.

- Changes: None.

2.4.4.2. Package “dataStructures”

- Requirements: *US-2-RQ-M-F1*, *US-2-RQ-M-F3;*
- Description: The packaging that contains all enums or data structure definitions used by the “messages” package.
- Goals: It provides an accurate and straightforward way of referencing collections of related data or declarations, as defined by the S2 Standard.
- Input: N/A.
- Output: N/A.
- Design: This package contains pure Typescript interfaces and enums that are either independent or interconnected with others contained within the same package. Thus, this package exists to support the “messages” one and establish fixed ways of referring to data within the various components it might be needed in.
- Changes: None.

2.4.5. Hooks

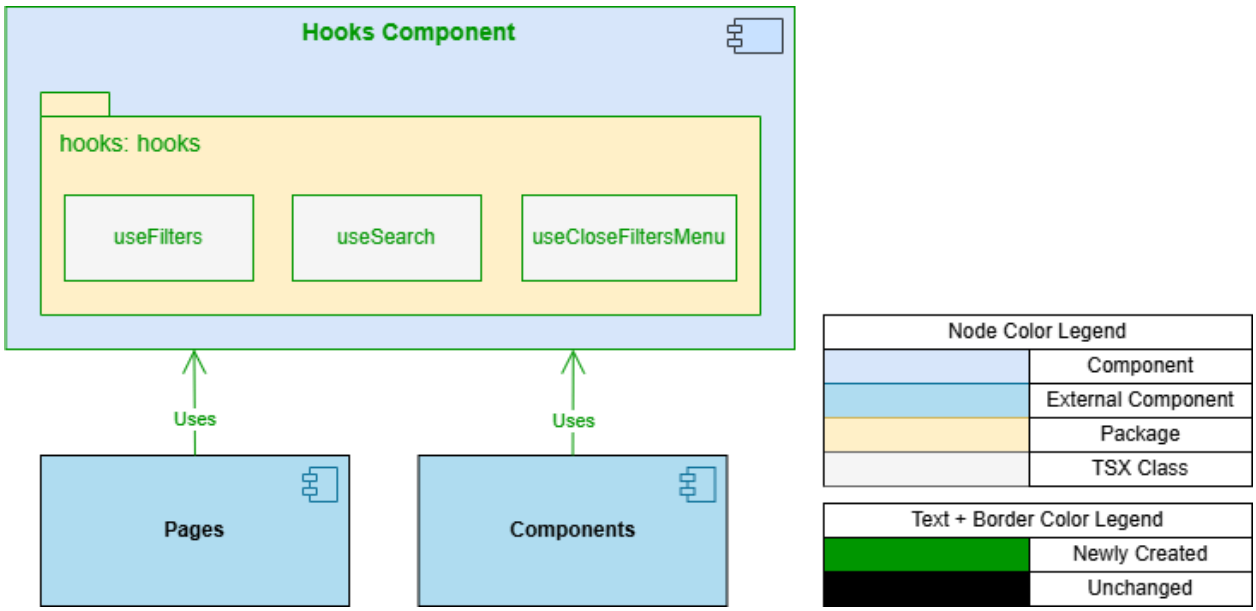

Figure 8: Single-Page Application's Hooks Module Diagram.

2.4.5.1. Package "hooks"

- Requirements: *US-9-RQ-M-F1*, *US-9-RQ-M-F2*;
- Description: A supporting component meant to separate simple classes from their more complex behavior.
- Goals: It aims to decouple and simplify the application, by extracting complex, possibly widely-used methods.
- Input: The first hook accepts a MessageHeader array and a Filters data structure (a sequence of booleans), the second hook accepts the same array along with a string messageID, and the third hook simply accepts a mouse-generated event.
- Output: The first and second hook return the filtered version of their inputted arrays after incorporating some logic, while the third hook removes a div that is no longer being used.
- Design: Hooks follow a specific React-imposed structure and are generally single-method implementations that are complex, but reusable, readable, and do not require any long-lasting variables to be stored. Thus, they simplify different components' logic, while providing a uniform way of executing specific processes that will possibly be used by other components in the future.
- Changes: The need for this component arose after refactoring the structure of the application that resulted from the first block. It was created as a way of ensuring the longevity, expandability, reusability, and flexibility of the system.

2.5. Class Diagrams

The following Class Diagrams abide by the standard UML notation, a brief explanation of which can be found both in the Glossary and on Figure 9, the latter of which also acts as the legend for Figures 10-13. Moreover, it is important to note that only the design of significant classes will be discussed, as to not overcrowd the document with self-explanatory details. For the same reason, the dependencies of all classes to the Models Component will not be graphed, but it can be assumed that every class who uses generics or non-Typescript native types, makes use of that component. Lastly, it is safe to assume that the deletions, refactorings, and additions indicated by the text and border colors of the figures contained in section 2.4, still propagate through this level of the application's architecture, as can be observed through their respective colors.

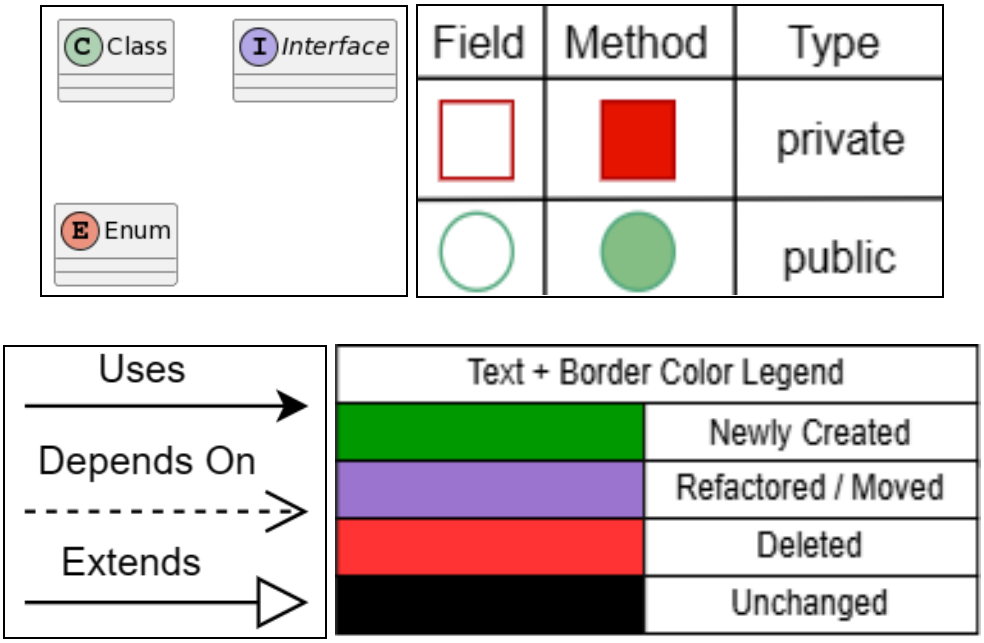

Figure 9: Legend for all Class Diagrams.

2.5.1. Packages “messages” and “popup”:

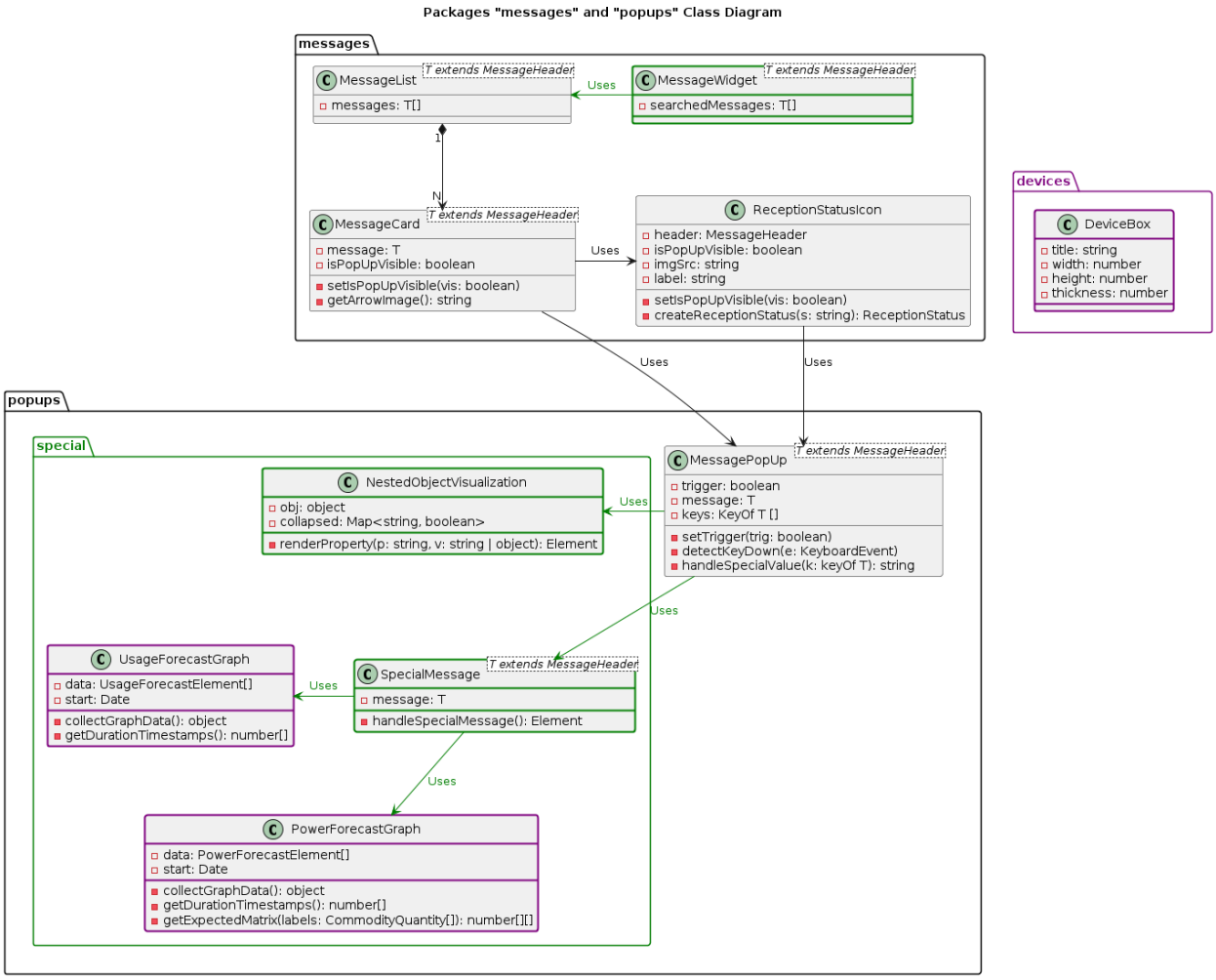

Figure 10: Class Diagram for packages “messages” and “popups” (including refactored “devices” package).

2.5.1.1. MessageWidget class

- Requirements: *US-1-RQ-M-F1*, *US-1-RQ-M-F2*, *US-1-RQ-M-F3*, *US-13-RQ-M-N1*;
- Description: The class responsible for rendering the entire CEM-RM communication in a visually appealing way.
- Goals: It aims to provide an additional level of abstraction to the MessageList class by decorating it with the functionality of the previously-separate DeviceBox class. Previously, this would be done directly within the Page class, but it was deemed better to decouple those classes and make rendering the MessageList simpler by creating a class responsible for this purpose.

- Input: It accepts an array of MessageHeaders from the Page class, which is assumed to contain all messages that need to be rendered, after all filters have been applied.
- Output: It renders a widget that contains the MessageCard representation of each MessageHeader passed to the MessageList's array.
- Design: This class utilizes a grid that splits the area of the renderable element into 12 rows and 12 columns. This provides a lot more customizability when placing the border of the MessageList and the boxes representing the CEM and RM respectively - something that was very limited when done via the DeviceBox class. Nonetheless, it uses the Decorator Pattern to wrap the MessageList class with this functionality before passing its MessageHeader array to the former. This provides a deeper Separation of Concerns within the "messages" package, as MessageWidget acts as the gateway into the entire package and its logic. Thus, the Page class can utilize it without having to worry about its underlying implementation, boosting its flexibility, customizability, and expandability.
- Changes: This class was created after the modernization of the application's UI which required splitting it into a grid in order to better and more precisely customize it. Thus, the DeviceBox class was incompatible with this change due to its rigidness, leading to its logic being incorporated into MessageWidget, which is also responsible for setting up the environment within which the MessageList will be rendered.

2.5.1.2. MessageCard class

- Requirements: *US-1-RQ-M-F2*, *US-1-RQ-M-F3*, *US-5-RQ-M-F1*, *US-5-RQ-M-F2*, *US-5-RQ-M-F3*, *US-6-RQ-M-F1*, *US-7-RQ-M-F1*, *US-10-RQ-M-F2*, *US-14-RQ-M-F1*;
- Description: The class responsible for rendering a singular MessageHeader, no matter if that came from a logged message or some backend warning.
- Goals: It provides a uniform way of displaying the common properties of all messages.
- Input: It accepts a MessageHeader instance or any of its children.
- Output: It renders the name of the message, an arrow denoting the direction the message was sent, the date-time it was received, and its reception status.
- Design: This is a simple class that takes advantage of generics in order to generalize its behavior for every type of message. In doing so, it utilizes a MessagePopUp and a ReceptionStatusIcon instance, initialized with the given message and its reception status property, respectively. Nonetheless, it uses the fields defined within the MessageHeader interface to render each message without delving into their unique properties, following these rules:

1. If the “message_id” property of the message is equal to null, then the message represents a backend log, so use the “line.png” image. Otherwise, if its “sender” property begins with “RM”, draw an arrow from the RM to the CEM and vice versa if not. This rule extends the usual S2 Standard definition, since no message under the FRBC Control Type is allowed to have a null “message_id” property, but is justified since the backend warnings of the S2 Analyzer backend do not belong in the standard.
2. If the “message_id” property of the message is equal to null, color its name red instead of blue, since it must represent a backend log.
3. Position the rendering of the time the message was sent near the device it was sent from. If no sender was specified, render it towards the CEM.

- Changes: As per requirement *US-14-RQ-M-F1*, all images used throughout the MessageCard class (such as the directional arrows/lines and the images used by the ReceptionStatusIcon class that represent the different reception statuses) were given descriptive “alt” labels in order to become screen-reader readable.

2.5.1.3. MessagePopUp class

- Requirements: *US-2-RQ-M-F1*, *US-2-RQ-M-F2*, *US-13-RQ-M-N2*;

- Description: The class responsible for rendering the properties of all messages and graphing their elements if needed.

- Goals: It provides a uniform way of representing the properties of each message, while allowing for customization for specific types of messages.

- Input: It accepts from its parent MessageCard, the message instance that will be rendered, a boolean denoting if the pop-up is visible, and a setter method for that boolean. Since the last two properties are shared with its parent component, closing the pop-up via its button or by clicking on its MessageCard a second time will both result in the pop-up closing.

- Output: It renders a draggable pop-up with all the properties of a given message in a property-value table. If the messages are “forecast” their elements are graphed underneath this table.

- Design: This class makes use of React’s “react-draggable” library in order to create a draggable pop-up that contains a table, with each row denoting the name and the value of each of the message’s properties. As such, it is able to customize how specific values are rendered by calling the “handleSpecialValue(k: keyOf T)” method which truncates long ReceptionStatus values or graph-related arrays and prints object-typed ones in their JSON representation. Moreover, by using the SpecialMessage class, it is able to compare the “message_type” value of its given message in order to assess whether to invoke a

specialized component, such as the graph-specific classes. On the other hand, if the message's table is too big or nested to be readable in a message pop-up, it uses the NestedObjectVIsualization class that uses recursion to build a collapsible property-value table instead. Thus, rendering both special values and messages is handled via a simple, expandable, and maintainable manner.

- Changes: Instead of MessagePopUp having to manually check the passed message's type and call the respective special visualization component, this functionality was extracted to the SpecialMessage class. This greatly simplifies the MessagePopUp class, reduces its coupling with the newly created "special" package, and provides a clear template for adding new specialized visualization options for different types of messages - boosting the expandability of the system.

2.5.1.4. SpecialMessage class

- Requirements: *US-2-RQ-M-F2*;
- Description: The class responsible for checking the type of a passed message and deciding which specialized visualization class would be best suited for rendering its properties.
- Goals: It aims to simplify the MessagePopUp class by decoupling it with the "special" package, while providing an easily expandable and comprehensible way of possibly adding more specialized message renderings in the future.
- Input: It accepts a MessageHeader instance that it can use to examine its "message_type" property and then extract the fields needed to be passed on to the special visualization classes.
- Output: Its output is solely dependent on that of the other classes in the "special" package it can refer to. If none match, then it returns nothing.
- Design: This class follows the Factory Pattern by using the "message_type" of its passed MessageHeader instance in order to decide which class of the "special" package should be instantiated and with what properties it should be initialized. Thus, expanding its selection is as easy as adding another condition to its "handleSpecialMessage()" method and creating a class within the "special" package that handles that type of message's rendering. This acts as a way of future-proofing the application by providing an extendable and comprehensible way of managing customized message renderings - an aspect of the application that is the most probable of being expanded upon in the future.
- Changes: The "handleSpecialMessage()" method along with any dependencies it was associated with, were extracted from the MessagePopUp class into the SpecialMessage one and all associated classes were aggregated into a single subpackage, namely "special".

2.5.1.5. UsageForecastGraph and PowerForecastGraph classes

- Requirements: *US-2-RQ-M-F2*;
- Description: The classes responsible for graphing the “elements” property of the UsageForecast and PowerForecast messages, respectively.
- Goals: It utilizes pre-existing libraries in order to graph these properties in a visually appealing way.
- Input: They both accept a starting date (“start”) and a UsageForecastElement or PowerForecastElement array (“data”), respectively, that represent an upcoming forecast.
- Output: Depending on the type of the data, these classes render a stepped line graph of the upcoming usage or power forecast of the device, as denoted by their “data” property.
- Design: Both classes utilize React’s “react-chartjs-2” library in order to pass data to and customize their graphs. For the latter class, when multiple lines are drawn in the same graph, they are all represented by different colors alongside an interactable legend.
- Changes: After the introduction of the SpecialMessage class, these classes along with the NestedObjectVisualization one were aggregated into the “special” subpackage due to their related concern of altering the appearance of specific message’s pop-ups.

2.5.2. Packages “parser”, “terminal”, and “navbar”:

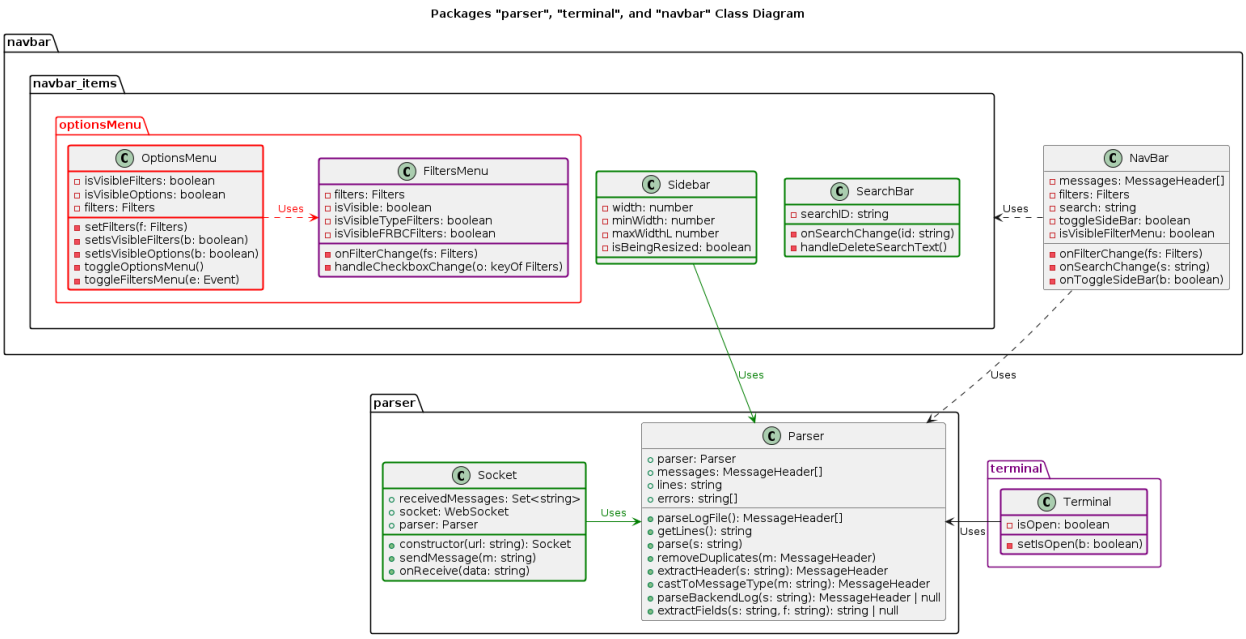

Figure 11: Class Diagram for the Parser Component and the “navbar” and “terminal” packages.

2.5.2.1. NavBar class

- Requirements: *US-7-RQ-M-N2*, *US-9-RQ-M-F1*, *US-9-RQ-M-F2*, *US-10-RQ-M-F3*, *US-11-RQ-M-F1*, *US-11-RQ-M-F2*;
- Description: The class responsible for rendering the application's navigation bar.
- Goals: It provides a way to fill up this navigation bar by easily-interchangeable buttons that offer the application's features.
- Input: It is able to register user-generated events, such as mouse clicks, on the different buttons it contains. It also accepts three setter methods from its parent Page class that allow it to transfer filter and search related data back to it in order to be used for processing by utilizing the Hooks component.
- Output: It renders the navigation bar of the application and any menus that stem from it.
- Design: Due to React's one-way data flow between components, this class relies on setter methods in order to pass data back, since instead of a traditional navigation bar that allows the user to traverse the different pages of the application, this one lays out all the functionality of the frontend to the user. As such, it contains buttons that allow them to load a ".txt" file or toggle the filter-menu, which after interacting with the user, causes some data to be sent to the navigation bar, which is returned to the encompassing Page class via these setters. Thus, it is easy to exchange these buttons by any other implementations, as long as they return what is required from them. The reason this class is depicted as "depending" on the other classes within its packages, instead of simply "using" them, is due to the fact that the introduction of additional buttons or the alteration of currently-used ones could lead to changes in the NavBar class via the introduction of additional properties it will possibly need to relay back to the Page class.
- Changes: With the addition of the *US-10-RQ-M-F3*, *US-11-RQ-M-F1*, *US-11-RQ-M-F2* requirements, the buttons "Pause Real-Time", "Load File", and a sidebar-toggling button were added to the navigation bar to handle each feature respectively. In addition, requirement *US-9-RQ-M-F2* led to the addition of a search bar that is rendered by expanding on the "Combobox" component offered by React's "headlessui" library, where the user can type the ID of a specific message. In that case, the SearchBar class accepts the inputted string and passes it to the NavBar which uses its "onSearchChange(s: string)" property to pass it back to its parent Page class in order for the data to be used for processing by the Hooks component.

2.5.2.2. Terminal class

- Requirements: *US-8-RQ-M-F1*, *US-8-RQ-M-F2*, *US-13-RQ-M-N1*, *US-13-RQ-M-N2*;

- Description: The class responsible for toggling a terminal-like pop-up containing the unprocessed logs of the given ".txt" file or the one being constructed in real-time.
- Goals: It offers a way to compare the actual logs with their frontend representations.
- Input: The string form of the inputted ".txt" file, as retrieved from the Parser class.
- Output: It renders a draggable and scrollable pop-up that contains the up-to-date raw logs of the given ".txt" file.
- Design: This class uses specific TailwindCSS labels in order to render an accordion-like terminal that can be expanded or collapsed on the bottom of the screen. Other than that, it is a very simple class that uses the lines of the ".txt" file as input. It is important to note that after its separation from the NavBar class, these lines are given to it by directly calling the Parser's "getLines()" method, meaning that the Terminal class is only loosely coupled with the Parser - making it reusable enough for rendering any pre-formatted string input.
- Changes: This class, along with its encompassing "terminal" package, were moved from "navbar.navbar_items" after its physical relocation on the Page's grid instead of being offered as a toggleable button. This way, the NavBar class stopped being the "middleman" between this and the Parser class, reducing its coupling and boosting its readability and maintainability. Lastly, per the client's request, the Terminal class stopped using React's "react-terminal-ui" library in order to render its terminal-like appearance and, instead, its appearance was customized using TailwindCSS labels.

2.5.2.3. Sidebar class

- Requirements: *US-11-RQ-M-F2*;
- Description: The class responsible for listing all the parsing errors that occurred while using the application, without allowing the system to break because of them.
- Goals: It increases the transparency between the user and the application, by providing them with a way of seeing exactly what went wrong during the visualization of the different messages.
- Input: It accepts an array of strings, where each cell represents one found parsing error.
- Output: It renders a scrollable and expandable sidebar menu that contains the exact parsing errors.
- Design: This class gets its contents by directly calling the Parser class' "getErrors()" method that returns an array of all caught parsing-related exceptions along with the line number they occurred at and a customized message from the Parser. Thus, if this array is not empty, it

maps every cell to a “<pre>” HTML element in order to retain their formatting, else it outputs “None.”. In addition, it uses React’s “useEffect” hook in order to register mouse-dragging on its rightmost side, indicating that the user wants to expand the width of the sidebar - something that is possible only up to half the screen. Nonetheless, the visibility of this class is dependent upon its respective button on the navigation bar, while the class itself is being directly used by the Page class too in order to fit it into its grid layout.

- Changes: This class was introduced during the second block’s development in order to cater to requirement *US-11-RQ-M-F2* specifically.

2.5.2.4. Parser class

- Requirements: *US-2-RQ-M-F3*, *US-5-RQ-M-F3*, *US-6-RQ-M-F2*, *US-8-RQ-M-F2*, *US-10-RQ-M-F1*, *US-10-RQ-M-F2*, *US-10-RQ-M-F3*, *US-11-RQ-M-F1*;
- Description: The class responsible for reading and parsing a “.txt” file or individual lines from it.
- Goals: It aims to provide the input of multiple other classes, either directly (e.g. Sidebar and Terminal) or indirectly (e.g. MessageWidget or MessageList) through its public methods.
- Input: A “.txt” file selected by the user after invoking a file explorer window or the lines of logged messages sent from the backend in real-time.
- Output: It does not render anything, but simply returns the string form of the selected file, its extracted MessageHeader array, and the array of parsing errors that occurred, when requested.
- Design: It is a pure Typescript class that follows the Singleton Pattern and provides an entrypoint to its private fields via the three public methods discussed in section 2.3.3. Apart from that, it is important to note that this class is dependent on every message model, since it instantiates and fills them in, according to the backend’s logs. Thus, extending this class with a new type of message to parse, is as simple as adding a new line to the switch statement of its “castToMessageType(m: line)” class, that accepts a line of the file and returns the message it contains by reading its JSON representation and then casting it to its respective frontend message model. Thus, it acts as a Factory Pattern implementation when it comes down to instantiating both messages via the aforementioned method or backend warnings’ MessageHeader representations via its “parseBackendLog(s: string, t :string)” method.
- Changes: During the second block’s development, the Parser class was extensively unit tested in order to ensure its correctness. As such and because of the inherent limitations of Jest (Javascript testing framework), all tested fields and methods of this class that were

previously private, had to become public in order to be used in the unit tests. Furthermore, since the Socket class represents the other half of the frontend's logic that needed to be unit tested, it was similarly affected. However, since this is a special usage case of these classes that does not naturally come up during the application's lifecycle, it is important to note that Figure 11.1 showcases the way these fields and methods are actually used throughout the application's runtime.

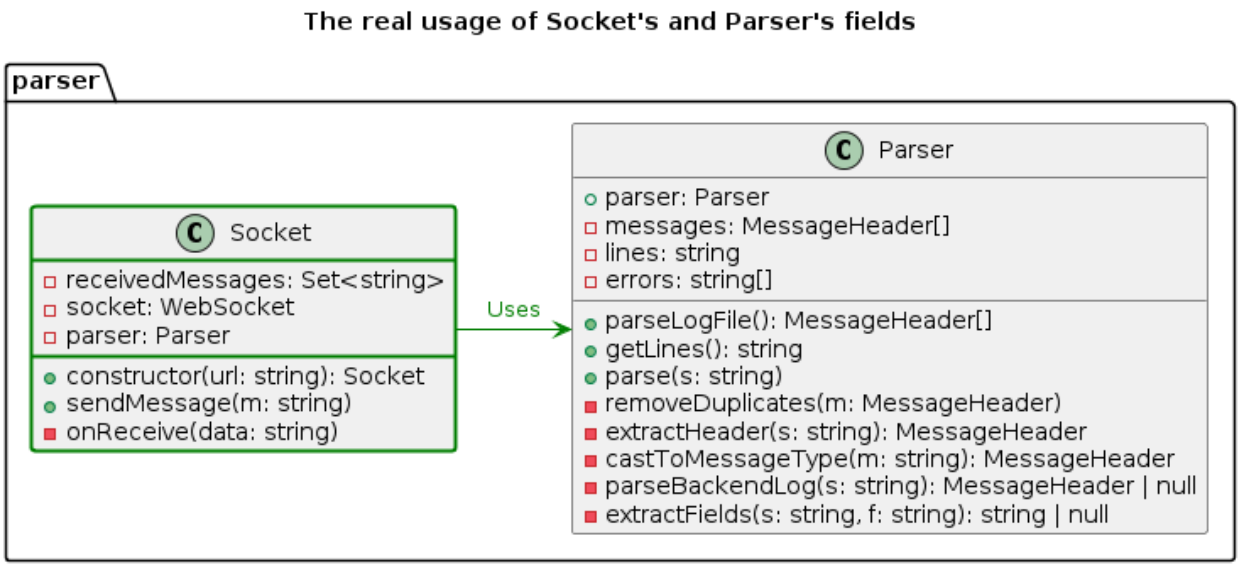

Figure 11.1: The real usage of the Socket and Parser classes' fields.

2.5.2.5. Socket class

- Requirements: *US-8-RQ-M-F2*, *US-10-RQ-M-F1*, *US-10-RQ-M-F2*;
- Description: The class responsible for establishing a connection to the S2 Analyzer backend and asynchronously passing any incoming messages to the Parser.
- Goals: It connects the frontend to the backend in order to form a coherent application.
- Input: In its constructor, it accepts a string "url" that is directly used for opening a WebSocket and connecting to that address. After that, this class accepts input in the form of invocations of its "onReceive(data: any)" method that receives a line of the connection history ".txt' logs before it is written to the file.
- Output: It does not render anything, but simply passes data to the Parser class by calling its "addLines(l: string)" and "parse(s: string)" methods. Also, it extensively logs about the status of the WebSocket's connection.

- Design: This is a very simple class, containing only a constructor and two methods, namely “onReceive(data: any)”, used for receiving messages or logs from the backend, and “sendMessage(s: string)”, used only once at the start of the frontend-backend communication in order to establish the health of the WebSocket’s connection. Apart from making sure that no exact message duplicates are received (hence its “receivedMessages” field being a Set), It does not do any kind of processing over the data it receives, leaving it up to the Parser to determine if it is worth processing.

- Changes: This class was introduced after the Dockerization of the frontend that made the process of debugging and developing it much easier. Thus, it materializes all requirements that have to do with the real-time rendering of exchanged messages along with making sure that all received lines are prepended with their correct timestamp and are concatenated to the Parser’s “lines” field.

2.5.3. Packages “pages” and “hooks”

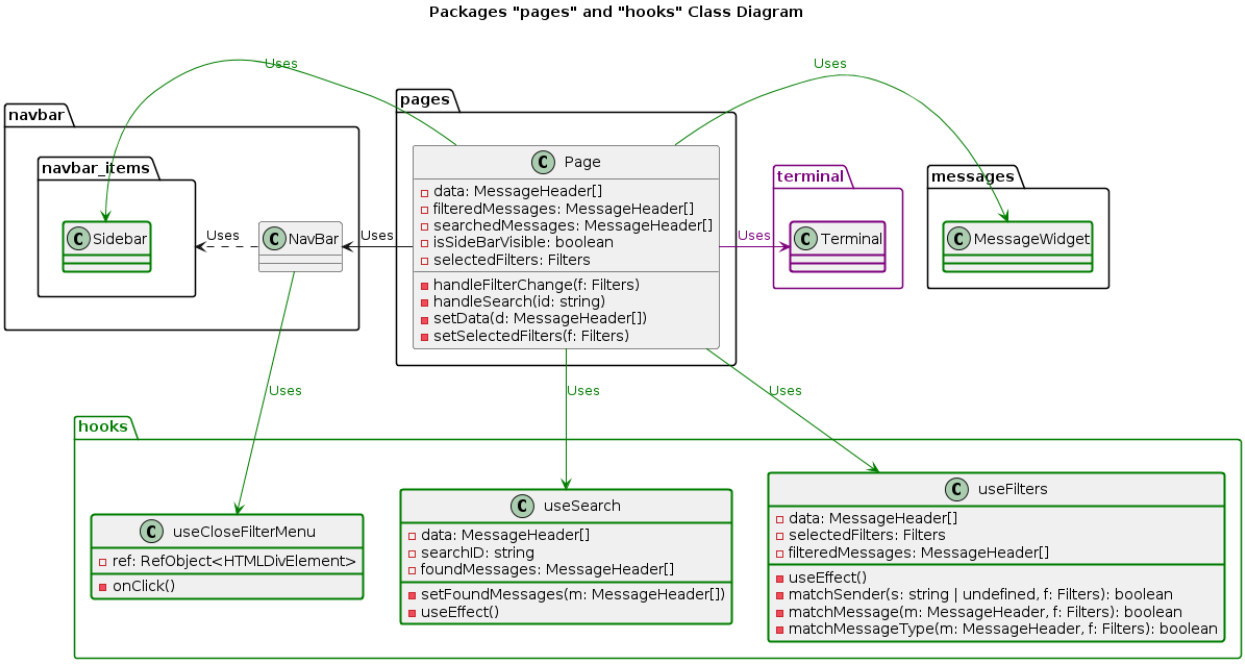

Figure 12: Class Diagram for the Pages and Hooks Components.

2.5.3.1. Page class

- Requirements: *US-1-RQ-M-F1*, *US-1-RQ-M-F2*, *US-3-RQ-M-N1*, *US-3-RQ-M-N2*, *US-4-RQ-M-N1*, *US-7-RQ-M-N1*, *US-12-RQ-M-N1*, *US-13-RQ-M-N1*, *US-13-RQ-M-N2;*

- Description: It is the class responsible for rendering the main page of the Single-Page Application.

- Goals: It provides the functionality of the Single-Page Application to the user by collecting independent components.
- Input: It does not accept any input, apart from the data it is able to exchange with other components via setter methods.
- Output: It renders the environment in which all the application's functionality is available and rendered to the user.
- Design: With the exception of the Sidebar class that was explained in section 2.5.2.3, the Page class only utilizes classes that act as entry points to specific functionality of the application: Firstly, it uses the NavBar to render its navigation bar and all of its associated buttons. Secondly, it uses the MessageWidget class to elegantly display the array of MessageHeaders that the Parser has extracted from the given ".txt" file. Thirdly, it uses the Terminal class in order to fit it into its grid layout, allowing the user to constantly view the exact lines being parsed by the frontend. Lastly, it utilizes two custom hooks defined within the Hooks component in order to handle the logic of implementing the search and filtering capabilities of the application onto its MessageHeader array. As such, it does not provide any additional functionality to any of these classes, but instead exists to coordinate them and to host the main MessageHeader array that will be filled with all parsed messages and propagated within the "messages" package.
- Changes: As briefly mentioned before, one big change to the Page class was that it was split into a 12 by 12 grid in order to more precisely and efficiently allocate screen space to its most important components. Furthermore, complex behavior, crucial to handling the message searching and filtering capabilities of the frontend was extracted into custom React hooks that reside within the Hooks component. This stands to modernize the codebase, make it easily testable, and boost its maintainability and expandability.

2.5.3.2. useCloseFilterMenu, useSearch, and useFilters classes

- Requirements: *US-9-RQ-M-F1, US-9-RQ-M-F2;*
- Description: The classes responsible for handling the complex logic behind applying the stackable filters and search capabilities of the frontend.
- Goals: Each class implements a hook aimed to handle a very specific function in a readable and easily testable and maintainable way.
- Input: Each class accepts specific properties associated with their behavior, similar to passing arguments to method calls.

- Output: None of these classes render anything to the user, but are used for internal computations instead.
- Design: In React, hooks are special functions that provide a gateway to specific React features such as state management, lifecycle methods, and functional components' behavior. Thus, all such classes follow a specific structure and naming convention, starting with the prefix "use" to ensure consistency. Nonetheless, hooks allow developers to extract and reuse logic across multiple components, promoting cleaner and more maintainable code. This cohesive approach enhances the readability and reusability of component logic in React applications, as seen by the simplification of the Page class. Thus, if in the future any other components need to filter messages (useSearch and useFilters) or remove a specific HTML element after the user clicks away (useCloseFilterMenu) from it, they can use the existing implementations in the Hooks component.
- Changes: As mentioned multiple times before, these classes were first established during the codebase maintenance and refactoring that took place in the second block.

2.5.4. Package "models.messages" and "models.dataStructures"

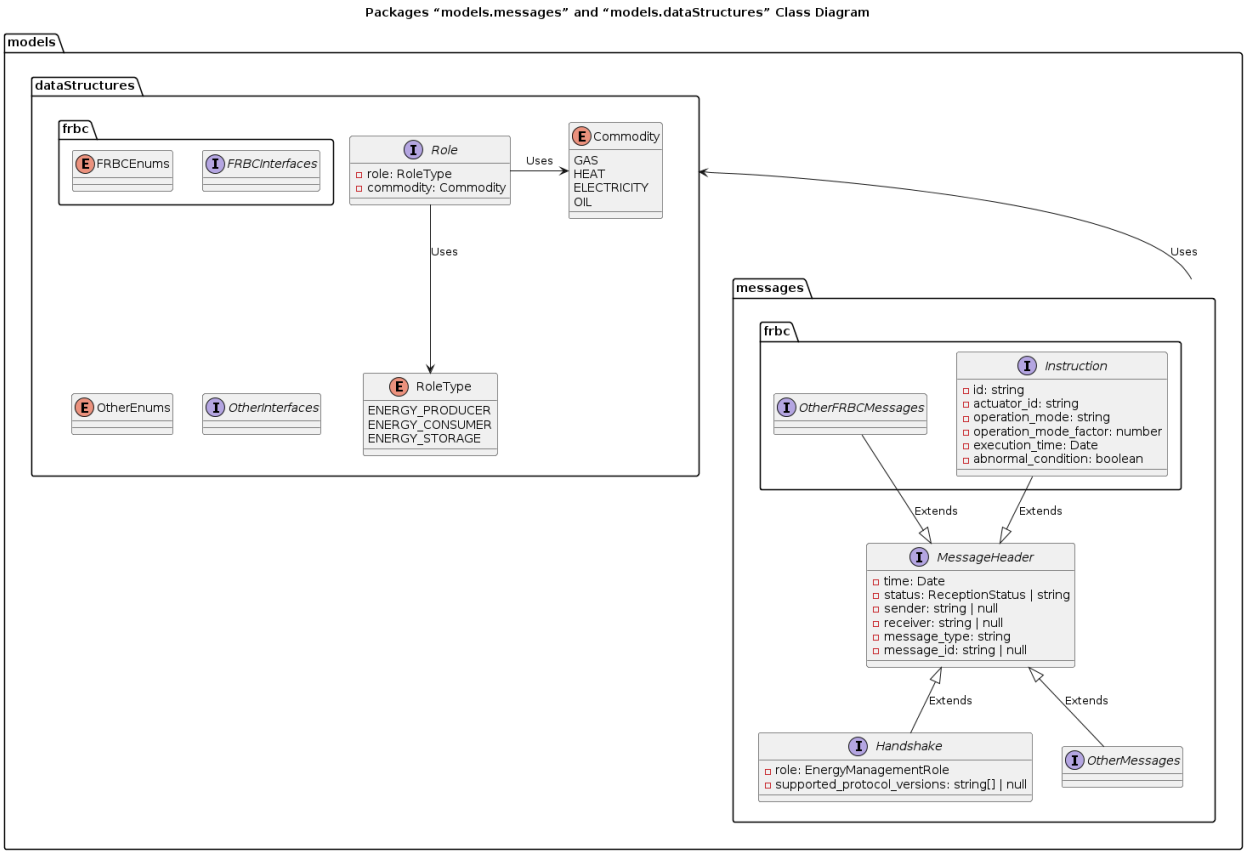

Figure 13: Class diagram for the Models Component.

2.5.4.1. MessageHeader and ReceptionStatus interfaces

- Requirements: *US-2-RQ-M-F1*, *US-2-RQ-M-F3*;
- Description: The former class contains all the common properties of messages while the latter contains a message-like data-structure that represents the ReceptionStatus message defined by the S2 Standard.
- Goals: They provide an accurate and complete description of the common properties defined within each message type, as well as allocating each ReceptionStatus message within the referenced message's "status" field.
- Input: N/A.
- Output: N/A.
- Design: The important design decision to note here is the relationship between the MessageHeader and the ReceptionStatus data-structure. After every message is sent, the receiver sends back to the sender a ReceptionStatus message, denoting the details about its acceptance. This idea was expanded upon by integrating these messages into the original one in order for it to be able to pass it onto the ReceptionStatusIcon class and render its status pop-up from the same MessageCard. However, in order to avoid a cyclic dependency between ReceptionStatuses and MessageHeaders, the former was reduced to its own data-structure, decoupling it from the latter and allowing for the easy transfer of reception-related data to their respective pop-ups.
- Changes: None.

2.6. Execution View

2.6.1. File Selection And Parsing Process

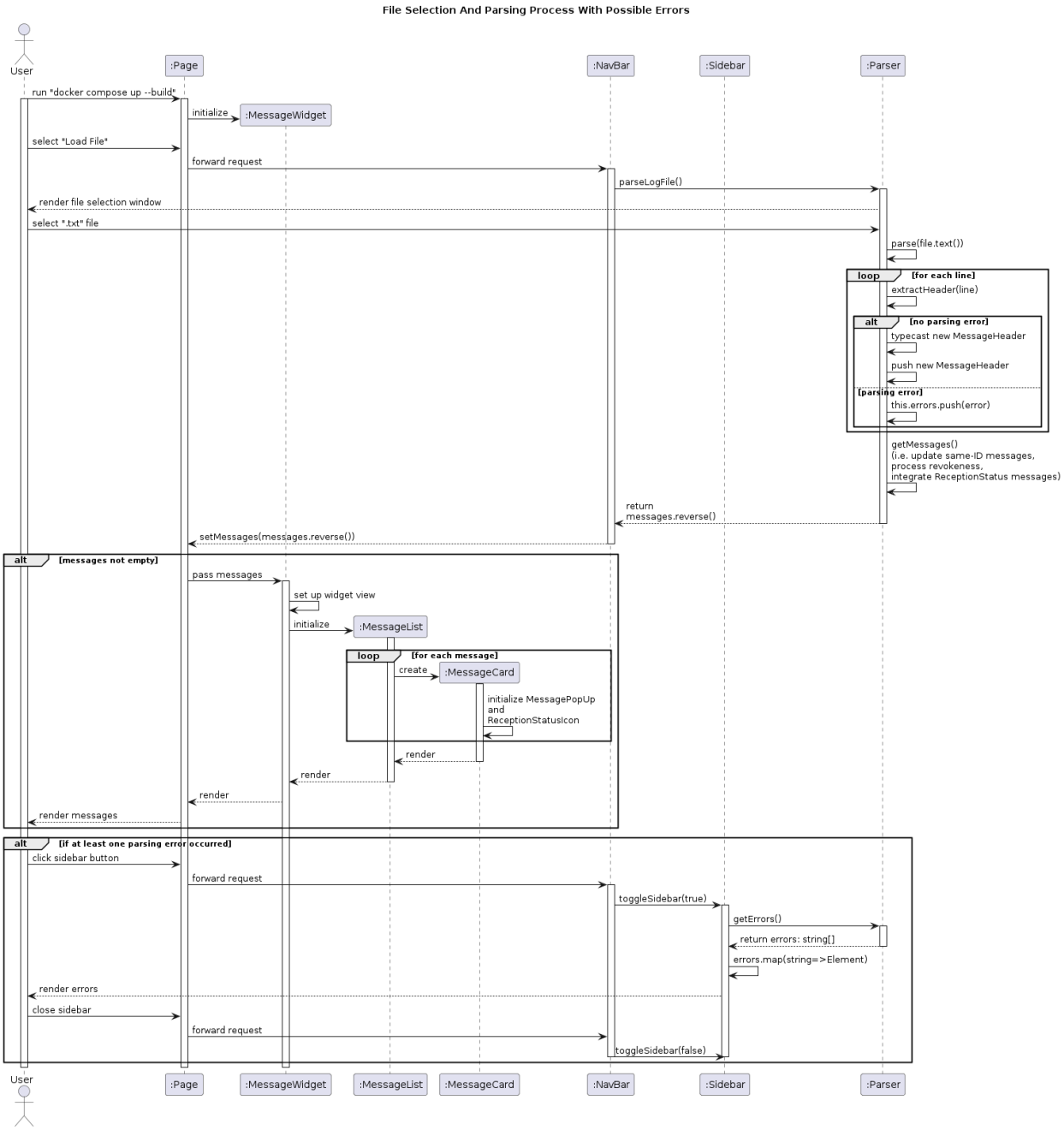

Figure 14: The file selection and parsing process Sequence Diagram.

- Requirements: *US-1-RQ-M-F1*, *US-2-RQ-M-F3*, *US-11-RQ-M-F1*, *US-11-RQ-M-F2*;

- Alternatives: As an alternative to this sequence, the Page class could have been responsible for rendering the file selection window, instead of allocating this responsibility to the Parser. On one hand, this would further promote the Separation of Concerns of these classes, but due to the one-way data-flow between React components, there could be some issues with the Page class knowing when to invoke the window. The way to solve this, would be for it to pass a setter method of a boolean flag to the Parser, which would invoke it whenever the file selection window was required. However, this would lead to an unnecessarily tighter coupling between the classes.
- Changes: None apart from the introduction of the MessageWidget and Sidebar classes.

2.6.2. Real-Time Message Rendering

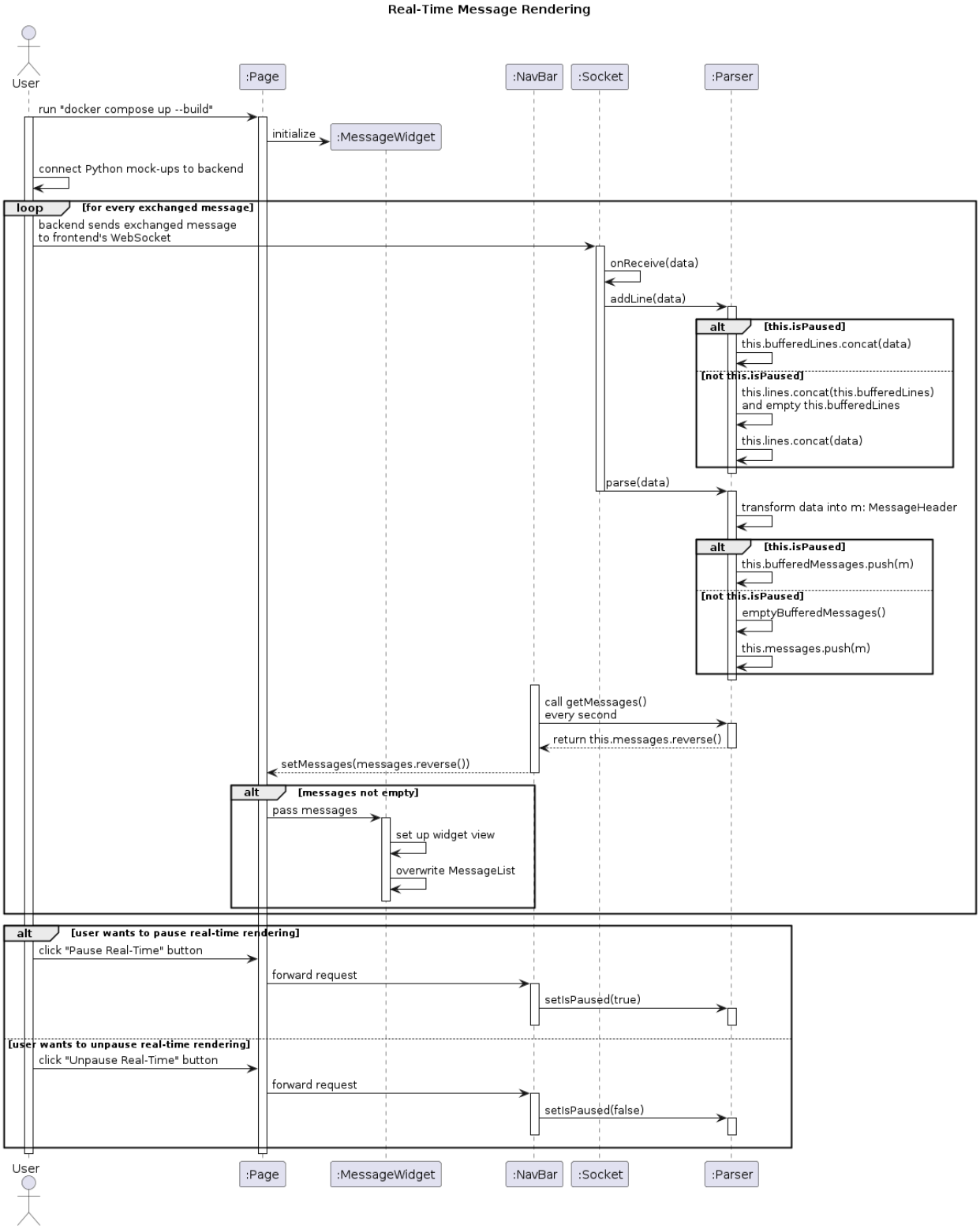

Figure 15: The real-time message rendering Sequence Diagram.

- Requirements: *US-1-RQ-M-F1*, *US-2-RQ-M-F3*, *US-10-RQ-M-F1*, *US-10-RQ-M-F2*, *US-10-RQ-M-F3*;
- Alternatives: Technically, the Page class could be responsible for continually polling the Parser, instead of the NavBar. However, since the NavBar already had a dependency on the Parser class, a setter for the Page's MessageHeader array as a property, and a general concern of setting the Page's MessageHeader array (through filters or the "Load File" button), it was deemed as the better fit. Thus, the Page class is kept simple and independent of the Parser - something that would otherwise significantly increase the coupling within the system, making it rigid and difficult to debug.
- Changes: This sequence was implemented after the introduction of the Socket class, which is the cornerstone of materializing the real-time message rendering capabilities of the frontend.

2.6.3. Interacting With Messages

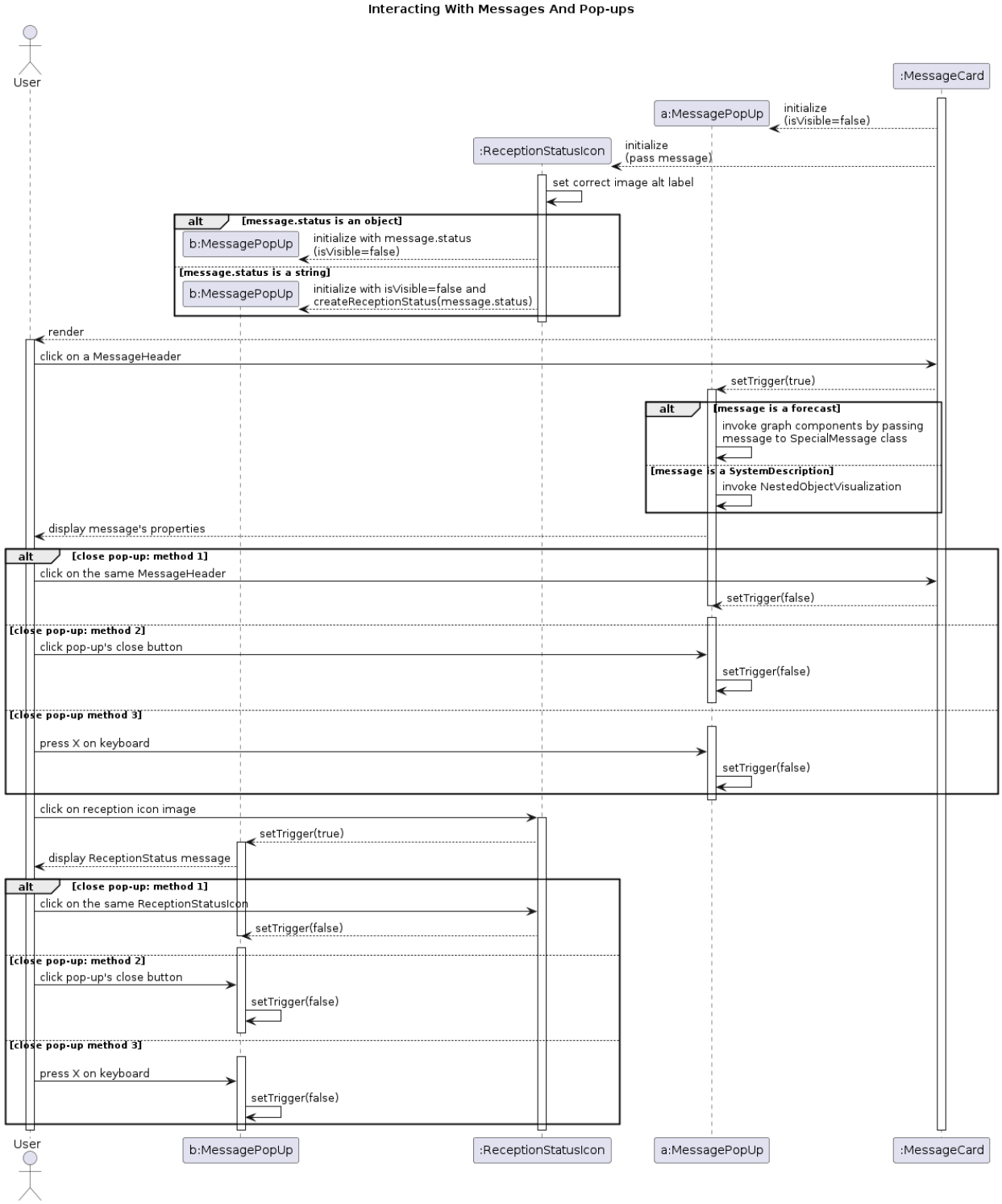

Figure 16: The Sequence Diagram for interacting with messages and their pop-ups.

- Requirements: *US-1-RQ-M-F1, US-1-RQ-M-F2, US-1-RQ-M-F3, US-2-RQ-M-F1, US-2-RQ-M-F2;*

- Alternatives: As an alternative to this sequence, there could be a shared MessagePopUp instance between the ReceptionStatusIcon and the MessageCard classes, which would get loaded with either the MessageHeader's or the ReceptionStatus' data when needed. On one hand, this would reduce the amount of object instantiation, but this would diminish the involved classes' Separation of Concerns by introducing additional dependencies between them.

- Changes: Two small new additions in the form of the keyboard shortcut "X" for quickly closing all pop-ups and the ReceptionStatusIcon class now being able to create a ReceptionStatus model for string status values.

2.6.4. Accessing The Terminal

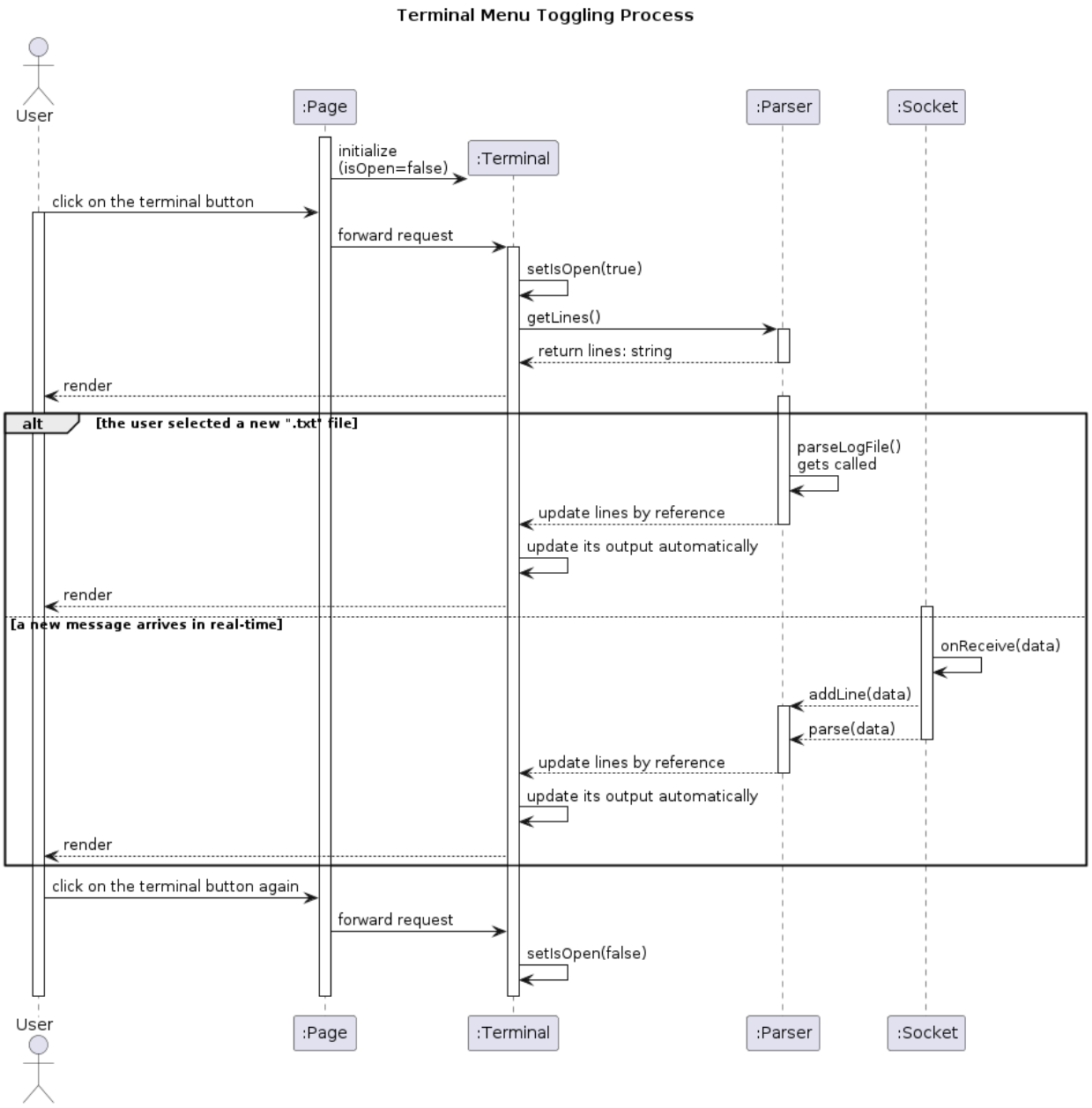

Figure 17: The Sequence Diagram for toggling the Terminal Menu.

- Requirements: *US-8-RQ-M-F1*, *US-8-RQ-M-F2*, *US-10-RQ-M-F1*;
- Alternatives: N/A.
- Changes: The addition of the real-time branch, along with the Socket class, and the decoupling of the Terminal and the NavBar class.

2.6.5. Filter Selection And MessageID Search

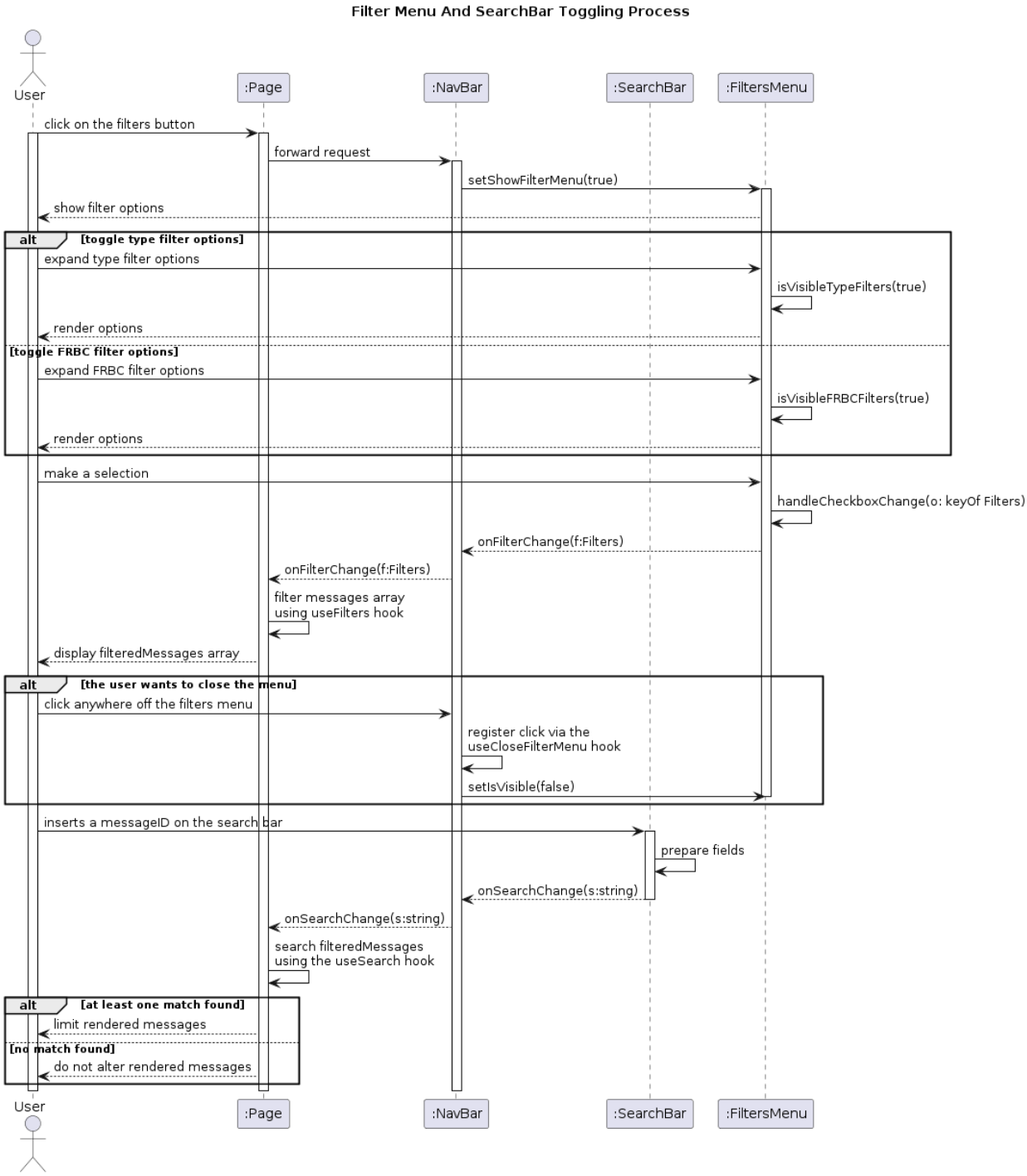

Figure 18: The Sequence Diagram for toggling the Filter Menu.

- Requirements: *US-9-RQ-M-F1*, *US-9-RQ-M-F2*;
- Alternatives: N/A.

- Changes: The removal of the OptionsMenu class and the introduction of additional functionality through the more extensive FiltersMenu options and the SearchBar class.

3. Technology Stack

3.1. Typescript

As outlined by requirement *US-3-RQ-M-N1*, the Visual and Debugging Applications must be written in Typescript, as requested by the client themselves. This was set due to its strongly typed nature, its ability for easier detection of compilation errors, its capabilities for seamless gradual integration and adoption with even Javascript codebases, and alignment with object-oriented programming (with classes, interfaces, and inheritance), making it a perfect fit for the project.

3.2. Frameworks

3.2.1. React

As outlined by requirement *US-3-RQ-M-N2*, the Visual and Debugging Applications must be developed using a modern frontend framework, the choice of which was up to the team. After consulting the client, they limited the team's choice between React and Vue. In order to decide, the team conducted the following comparison:

Feature	React	Vue	Source
Learning Curve / Simplicity	Could prove difficult to grasp for beginners	Beginner-friendly	[1]
Architecture	Flexible (used as a library or framework)	Lightweight	[2]
Community	Well-established and wide-reaching community	Relatively newer	[3]
Support	Wide variety of supporting libraries needed for the project	Fewer options	[4]
File organization	Divides files into components and pages (good for SPAs)	Incorporates CSS into each component's file	[5]

On one hand, Vue's comprehendible learning curve, lightweight architecture, and integration of HTML templates in its components, proved to be a more beginner-friendly environment. On other hand, React's simplicity, wide-reaching community, large variety of supporting libraries (for features related directly to the project, such as draggable components and graph rendering), and component-based structure fit well with the Single-Page Application nature of the project.

In the end, React was chosen as the most suitable framework due to its well-established ecosystem and all of the team having prior experience with it.

3.2.2. TailwindCSS

As outlined by requirement *US-13-RQ-M-N1*, the Visual Application should use a contemporary CSS framework in order to make managing its CSS files as easy as possible, while utilizing a modern look and feel. While considering possible options, TailwindCSS and MaterialUI came immediately to mind.

Feature	TailwindCSS	MaterialUI	Source
Learning Curve / Simplicity	Steep learning curve	Beginner-friendly	[6]
Customizability	Extensive customization options for class labels and	Pre-constructed components with limited customizability	[7]
Price	Completely free to use in full	More advanced features are locked behind a paywall	[8,9]
Community	Well-established and wide-reaching community	Well-established and wide-reaching community	[10]
Flexibility	Can be user with any framework and allows the creation of customized class labels, plugins, etc	React-only, but allows for other CSS management frameworks to be used on top of it, even though this may become complicated	[11]

As observed, while TailwindCSS provides a wide selection of CSS options that, while having a steep learning curve, allow for extensive and detailed customization of the frontend, the latter allows the frontend to use pre-constructed components that come with their own styling, modeled after Google's UI. However, TailwindCSS's utility-first principle and flexibility were important factors during the development of the project, while MaterialUI's pre-established styling may prove to be limiting and difficult to expand in the future.

In the end, TailwindCSS was chosen as it was deemed more suitable for the project for the reasons described above.

3.2.3. Jest

As per the *US-3-RQ-M-N1* requirement, no matter what frontend framework was chosen, the application must be written in Typescript. Thus, the team had to select a testing framework to use for establishing the unit tests of the Parser and Socket classes that worked well with both Typescript and the frontend framework that ended up being chosen.

Feature	Jest	Mocha	Jasmine	Source
Complexity	Simple for beginners	Steep initial learning curve	Simple for beginners	[12]
Suitability	Optimally paired with React	Optimally paired with Node.js	Optimally paired with Angular	[13]
Community	Most popular testing framework out of the three	Active and engaged community	Active and engaged community	[14]
Configuration	Simple configuration	Complex configuration	Simple configuration	[15]
Flexibility	Works for both Javascript and Typescript applications natively	Complex configuration process to make it work with non-Node.js apps	Works for most frameworks and libraries (needs some configuration for Typescript)	[16]

Thus, Jest was selected due to its simplicity, good performance, and easy integration with Typescript, offering features such as zero configuration setup, a user-friendly API, and built-in mocking and assertion libraries for testing Javascript and Typescript code. Also, Jest benefits include fast test execution with parallelization, snapshot testing for UI components, and an intuitive debugging mode - all of which are essential to testing any application.

3.3. Docker

As outlined by requirement *US-12-RQ-M-N1*, the client has specified that both the Visual and the Debugging Application must be deployable via Docker. As such, the S2 Analyzer backend and frontend images are deployed under the same container, alongside a websocket image that facilitates their communication. In addition, the client provided the team with some mock RM and CEM images in order to be used for testing the real-time message rendering feature of the frontend.

3.4. GitHub

As a repository hosting and version control system, GitHub offers crucial features to the development of the Visual and Debugging Applications, such as: branch management, version archival, issue and merge request handling. It was chosen over GitLab by TNO themselves, since they were already using GitHub for hosting their backend code. On top of that, there was no immediate need for GitLab's CI/CD or cloud-hosting features that would require switching

over. Nonetheless, it should be noted that the project is open-source and, thus, all of its repositories are public.

4. Glossary

API: Application Programming Interface - a set of rules and protocols that allows different software applications to communicate with each other.

CEM: Customer Energy Manager - a system responsible for orchestrating the energy usage and flexibility of devices by connecting to their Resource Managers in the context of energy management systems.

CSS: Cascading Style Sheets - a style sheet language used for describing the presentation of a document written in HTML or XML, referring to aspects such as layout, colors, and fonts.

DOM Tree: Document Object Model Tree - a hierarchical representation of the structure and content of an HTML or XML document. It models the document as a tree of nodes, where each node represents a part of the document, such as an element, attribute, or piece of text.

FRBC: Fill Rate Based Control - a sub-protocol within the S2 Standard that defines a set of messages for exchanging energy flexibility information between Customer Energy Managers and Resource Managers.

HTML: HyperText Markup Language - the language that defines the structure of web or web-browser traversable pages.

HTTP: Hypertext Transfer Protocol - an application protocol for distributed, collaborative, hypermedia information systems, commonly used for transferring data between a web server and a web browser.

JSON: JavaScript Object Notation - a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.

MVP: Minimum Viable Product - the initial proof of concept of the project, delivered midway through production.

RM: Resource Manager - a component within the energy management system responsible for managing the energy resources of devices in buildings or other systems.

S2 Consortium: A group of organizations collaborating to develop and promote the S2 Standard, a standard for exchanging standardized energy flexibility information between energy management systems and energy equipment in buildings.

S2 Standard: A standard developed by the S2 Consortium for the exchange of standardized energy flexibility information between Customer Energy Managers and Resource Managers.

SPA: Single Page Application - a web application or website that interacts with the user by dynamically rewriting the current web page with new data, instead of loading entire new pages.

TNO: Netherlands Organisation for Applied Scientific Research (Nederlandse Organisatie voor Toegepast Natuurwetenschappelijk Onderzoek) - an independent non-profit research organization involved in various scientific and technological fields.

TSX: TypeScript XML - a syntax extension for TypeScript infused with React elements to describe what the UI should look and behave like.

UML: Unified Modeling Language - a universal way of demonstrating the dependencies that occur inside a system. In this case, the following notations are used:
1) Continuous Line: Component A uses component B, but changing B does not affect A.
2) Dashed Line: Component A depends on component B, and changing B affects A.
3) Black-Diamond Line: Component A comprises many components B. If A is deleted, so are the B components it contains.
4) White Triangular Arrow: Component A extends B, meaning B is a child of A (near the arrow).

WebSocket: A communication protocol - it enables real-time interaction between a client and a server over a single, long-lived connection. It allows data to be sent and received simultaneously, reducing latency and improving performance for applications requiring constant data exchange, such as chat applications or live updates.

XML: Extensible Markup Language - a markup language that allows the transmission, storage, and reconstruction of data.